

SoK: Robustness in Large Language Models against Jailbreak Attacks

Feiyue Xu¹, Hongsheng Hu¹, Chaoxiang He¹, Sheng Hang¹, Hanqing Hu¹, Xiuming Liu¹, Yubo Zhao¹,
Zhengyan Zhou¹, Bin Benjamin Zhu², Shi-Feng Sun¹, Dawu Gu¹, and Shuo Wang¹

¹Shanghai Jiao Tong University, China

²Microsoft Corporation

Abstract—Large Language Models (LLMs) have achieved remarkable success but remain highly susceptible to jailbreak attacks, in which adversarial prompts coerce models into generating harmful, unethical, or policy-violating outputs. Such attacks pose real-world risks, eroding safety, trust, and regulatory compliance in high-stakes applications. Although a variety of attack and defense methods have been proposed, existing evaluation practices are inadequate, often relying on narrow metrics like attack success rate that fail to capture the multidimensional nature of LLM security. In this paper, we present a systematic taxonomy of jailbreak attacks and defenses and introduce Security Cube, a unified, multi-dimensional framework for comprehensive evaluation of these techniques. We provide detailed comparison tables of existing attacks and defenses, highlighting key insights and open challenges across the literature. Leveraging Security Cube, we conduct benchmark studies on 13 representative attacks and 5 defenses, establishing a clear view of the current landscape encompassing jailbreak attacks, defenses, automated judges, and LLM vulnerabilities. Based on these evaluations, we distill critical findings, identify unresolved problems, and outline promising research directions for enhancing LLM robustness against jailbreak attacks. Our analysis aims to pave the way towards more robust, interpretable, and trustworthy LLM systems. Our code is available at [Code](#).

1. Introduction

Large language models (LLMs) have rapidly become integral to a wide range of applications, including coding assistants [69], [78], healthcare advice [10], customer service [79], education [34], and scientific research [96]. Their strength stems from their ability to comprehend natural language, generate coherent and contextually appropriate responses, and adapt across diverse domains with remarkable flexibility. However, their growing adoption has sparked serious security concerns. In particular, malicious actors can jailbreak an LLM’s safety guardrails, coercing the model into producing prohibited or harmful outputs [55], [72]. Early demonstrations revealed that even seemingly simple strategies, such as role-playing as an unconstrained agent or embedding hidden instructions, could circumvent safety policies in widely deployed models like GPT-3.5-Turbo [93].

Building on these initial findings, jailbreak attacks have since diversified and become increasingly sophisticated. Beyond early role-playing tricks, researchers have developed techniques such as gradient-based prompt optimization [108], automated attack orchestration using auxiliary LLMs [12], [45], and adaptive strategies spanning multiple turns or modalities [57], [67], [70], [102]. As jailbreaks become more diverse and sophisticated, so too do their impacts: attacks now go beyond simple safety bypasses to produce contextually rich, targeted harmful outputs that enable more scalable and covert exploitation. In parallel, the community has proposed defenses such as adversarial training [60], instruction alignment through fine-tuning or unlearning [14], [100], and safety layers that filter outputs prior to delivery [31], [95].

However, despite rapid progress in attack and defense techniques, evaluations of their effectiveness and robustness remain fragmented, often lacking standardized benchmarks and reproducible protocols. Existing frameworks [11], [71] typically collapse performance into a single metric, attack success rate (ASR), while overlooking other critical dimensions such as stability across runs, cross-model transferability, and internal impact on model mechanisms. This one-dimensional perspective creates significant blind spots. For example, an attack may appear highly effective based on ASR yet fail to reproduce consistently, or two attacks with similar ASRs might exploit fundamentally different vulnerabilities, implying very different defense strategies. Such limitations risk misleading conclusions and impede the development of dependable safeguards. To address these gaps, a systematic study and a comprehensive evaluation framework are urgently needed to consolidate existing knowledge, illuminate trade-offs among current methods, and guide future research toward securing LLMs against jailbreak threats.

In this paper, we systematize the state of the art in jailbreak research for LLMs, providing a comprehensive landscape and taxonomy of attack strategies, defense mechanisms, and evaluation methodologies. Motivated by the limitations of existing benchmarks, such as their over-reliance on ASR and neglect of stability, transferability, and real-world considerations, we introduce Security Cube, a multi-dimensional evaluation framework designed to capture a fuller spectrum of jailbreak characteristics. Leveraging this framework, we benchmark representative attacks and defenses, uncovering key insights into current challenges

and highlighting opportunities for more robust and reliable LLM safety mechanisms.

Our main contributions are summarized as follows:

- **Comprehensive taxonomy of attacks and defenses.** We develop a systematic taxonomy that categorizes jailbreak attack and defense methods based on their dominant underlying mechanisms, ensuring clarity and minimal category overlap. Beyond the taxonomy, we derive key insights that provide a structured and comprehensive understanding of the jailbreak landscape.
- **Security Cube: a multi-dimensional evaluation framework.** We systematically review and compare existing evaluation benchmarks and identify critical gaps, such as their over-reliance on a single metric and narrow coverage across attack, defense, and judge types. To address these shortfalls, we propose Security Cube, a holistic evaluation framework that integrates 7 attack metrics, 3 defense metrics, and 4 judge metrics, several introduced and evaluated for the first time. This design facilitates rigorous, comprehensive assessment of jailbreaks and defenses, establishing a stronger foundation for improving LLM security.
- **Empirical benchmarking and key findings.** We apply Security Cube in a comprehensive study spanning 13 representative attacks, 5 defenses, and 4 automated judges across major categories. Our results uncover new vulnerabilities, reveal overlooked weaknesses, and demonstrate that multi-dimensional evaluations often contradict conclusions drawn from ASR alone.
- **Research directions for LLM safety.** Building on our empirical findings, we outline promising directions to advance jailbreak research and LLM security, including exploring new attack and defense paradigms, instituting continuous red-teaming, developing community-shared benchmarks, and deepening understanding of jailbreak interpretation for safety assurance.

2. Taxonomies of Attacks and Defenses

2.1. Problem Definition of Jailbreak

Let an LLM be a function $M(\cdot) : \mathcal{P} \rightarrow \mathcal{R}$, where $\mathcal{P}$ is the space of prompts and $\mathcal{R}$ is the space of responses. An LLM jailbreak is an adversarial procedure that aims to induce a safety-aligned model $M_a(\cdot)$ to violate its safety policies. Formally, given a harmful prompt P_{harm} that would normally be refused by the aligned model $M_a(\cdot)$, the attacker constructs a jailbreak prompt P_j through transformation $\mathcal{T} : \mathcal{P} \rightarrow \mathcal{P}$:

$$P_j = \mathcal{T}(P_{\text{harm}}).$$

The model’s response to this jailbreak prompt is then:

$$R_t = M_a(P_j).$$

To assess whether the jailbreak is successful, we follow prior works [2], [7], [11], [12] in employing a binary evaluation function $\text{Judge}(\cdot, \cdot) : \mathcal{R} \times \mathcal{P} \rightarrow \{\text{true}, \text{false}\}$, which returns

true if the model’s output R_t fulfills the malicious intent expressed in the original prompt P_{harm} . Formally, a jailbreak is deemed successful if:

$$\text{Judge}(R_t, P_{\text{harm}}) = \text{true}.$$

Note that P_j is not used as input to the Judge function, since the transformation $\mathcal{T}$ may significantly alter the surface form of the prompt, making it unsuitable for intent matching.

2.2. Threat Models

In this work, we systematically evaluate the robustness of LLMs against prompt-based jailbreak attacks by modeling a red-team adversary characterized as follows:

Adversary goals. The attacker aims to craft inputs that induce the model to violate safety policies and produce harmful content, while minimizing effort and maximizing attack generality across prompts, tasks, and models.

Adversary capabilities. We cover both black-box and white-box attack settings:

- **Black-box attackers** can access only the model’s input-output API, observe responses (including refusals), iteratively adapt prompts, and may use surrogate models.
- **White-box attackers** have full access to model internals (parameters, gradients, architecture) and can optimize adversarial prompts directly using the target model.

Defender capabilities. We also consider both black-box and white-box defense settings:

- **Black-box defenders** have access only to the model’s inputs and outputs. They can filter or modify the input before inference, and filter or adjust the output afterwards.
- **White-box defenders** have full access to the model internals. They can finetune the model or monitor its internal representations and parameters during inference.

Constraints and assumptions. We do not assume that the attacker has access to the training data, the ability to fine-tune the model, or the use of pre-filling techniques to elicit specific responses. This constraint ensures that the evaluation focuses on jailbreaks that arise from the model’s own behavior, rather than from external manipulations.

2.3. Attack Overview

To provide a structured view of the diverse landscape of red-teaming techniques against large language models, we propose a novel taxonomy of representative attack methods, summarized in Table 1. In this taxonomy, each category is defined by a distinct underlying mechanism. For attack methods that could plausibly span multiple categories, we classify them according to their dominant mechanism.

Attack taxonomy. Specifically, we categorize existing attack methods into seven types:

- **Logprobe-based attacks** refer to attacks that exploit exposed diagnostic signals, e.g., log outputs, auxiliary classifier scores, or (in white-box cases) gradient/activation information, to iteratively choose inputs that raise the chance the model produces disallowed content.

TABLE 1: Overview and comparison of existing jailbreak methods. **Code:** ✓ indicates open-source code is available; ✗ no open-source implementation. **Granularity:** *T* means harmful prompt is created at the token level; *P* at prompt level. **Access:** ● black-box query access to the target model; ● (S) black-box query access to the target model with a surrogate model; ○ full white-box access. **Feedback:** ● requires iterative feedback from the target model; ○ no interaction needed. **Requirements:** ⚙️ means attackers require to fine-tune or train a red-teaming model; 🧠 means attackers employ an LLMs to generate harmful prompts; ○ means attackers require only access to the target model. **Strategy:** ⚙️ means the harmful prompt is automatically generated by specific attack algorithms or LLMs; 👤 means the harmful prompt is mainly generated by human designed rules, e.g., template, with LLMs assisting in tasks like rewriting or role-playing. **Diversity** measures the complexity of variants in the harmful prompt. **Low** indicates the variants of the harmful prompt for a single goal is fewer than 10; **Medium** indicates moderate number of variants between 10-100. **High** indicates more than 100 variants, i.e, a broad and significant variation in wording, syntax, and attack logic.

Type	Attack	Reference	Year	Code	Granularity	Access	Feedback	Requirements	Strategy	Diversity
Logprob	GCG	[108]	2023	✓	T	○	●	⚙️	⚙️	Low
	AutoDAN	[46]	2023	✓	T+P	○	●	⚙️	⚙️	Medium
	AmpleGCG	[42]	2024	✓	T	○	●	⚙️	⚙️	Low
	ColdAttack	[25]	2024	✓	T+P	○	●	⚙️	⚙️	Medium
	DSN	[104]	2024	✓	T	○	●	⚙️	⚙️	Low
	MAC	[98]	2024	✓	T	○	●	⚙️	⚙️	Low
	PAL	[73]	2024	✓	T	● (S)	●	⚙️🧠	⚙️	Low
	LLM-Adaptive	[2]	2024	✓	T+P	● (S)	●	⚙️🧠	⚙️	Medium
Shuffle	DualBreach	[27]	2025	✗	T	● (S)	●	⚙️🧠	⚙️	Low
	BON	[28]	2024	✓	T	●	●	🧠(J)	⚙️	Medium
LLM	Flip	[48]	2025	✓	T	●	○	○	👤	Low
	PAIR	[12]	2023	✓	P	●	●	🧠	⚙️	High
	TAP	[52]	2023	✓	P	●	●	🧠	⚙️	High
	AutoDAN-Turbo	[45]	2024	✓	P	●	●	⚙️🧠	⚙️👤	High
	JailPo	[37]	2024	✗	P	●	●	🧠	⚙️	High
	ArrAttack	[38]	2025	✗	P	●	○	🧠	⚙️	High
Strategy	Auto-RT	[47]	2025	✓	P	●	○	⚙️🧠	⚙️👤	High
	ReNeLLM	[18]	2023	✓	P	●	●	🧠	👤	Medium
	PAP	[94]	2024	✓	P	●	○	🧠	👤	Medium
	DAN	[72]	2024	✓	P	●	○	○	👤	Low
	CodeAttacker	[66]	2024	✓	P	●	○	○	👤	Low
	QueryAttack	[109]	2025	✓	P	●	○	○	👤	Medium
	DrAttack	[39]	2024	✓	P	●	●	🧠	👤	Medium
Multi-round	DIE	[84]	2025	✗	P	●	●	🧠	👤	Medium
	ICRT	[87]	2025	✓	P	●	●	🧠	👤	Medium
	DeepInception	[40]	2023	✓	P	●	●	🧠	👤	Medium
	Crescendo	[70]	2024	✗	P	●	●	○	👤	High
	MRJ	[77]	2024	✗	P	●	●	🧠	👤	High
	ActorBreaker	[67]	2024	✓	P	●	●	🧠	⚙️	High
Flaw	Tempest	[102]	2025	✓	P	●	●	🧠	⚙️	High
	CipherChat	[93]	2023	✓	P	●	○	○	👤	Low
	Multijail	[16]	2023	✓	P	●	○	○	👤	Low
Template	ArtPrompt	[33]	2024	✓	P	●	○	○	👤	Low
	GPTFuzzer	[91]	2023	✓	P	●	●	🧠	👤	Medium

- **Shuffle-based attacks** try to evade safety by making harmless textual edits, e.g., reordering tokens, inserting separators or extra whitespace, or other small surface-level perturbations, so the prompt remains semantically intact for humans but changes how the model parses or responds.
- **LLM-based attacks** refer to attacks that use other LLMs as the primary means to automatically create, paraphrase, or iteratively refine prompts with the goal of increasing the chance a target model will produce disallowed content.
- **Multi-round-based (conversational) attacks** use multiple dialogue turns to gradually steer the model into producing disallowed content by building context, lowering guard signals, or exploiting state carried across turns.
- **Flaw-based attacks** exploit specific, model-dependent weaknesses, e.g., quirks in multilingual handling, failure modes from pretraining data gaps, or systematic miscalibrations, to craft inputs that reliably cause unsafe outputs.
- **Strategy-based attacks** refer to attacks that rely primarily on human-designed manipulative strategies, e.g., social, procedural, or rhetorical devices, to coax the model into producing disallowed content. For example, attackers use persuasive or procedural structure (framing, step requests, or persona appeals) to make an unsafe outcome seem like a legitimate, harmless task.
- **Template-based attacks** start from a set of complete, pre-defined prompts and then generate many variants by applying controlled mutations (e.g., phrase permutation). The attacker searches these mutated versions to find prompts that bypass safety.

For each attack method we summarize key features (granularity, access, feedback, requirements, strategy, diversity) in Table 1, producing a taxonomy of red-teaming (e.g., manual vs. automated) that clarifies core mechanisms and enables systematic comparison of strengths and limitations.

Key observations. From Table 1, we gain the following key

observations of the current jailbreak attack landscape:

① **Logprob, Strategy, and LLM-based attacks dominate the LLM jailbreak landscape.** When categorized by method, most jailbreaks since 2023 cluster into three classes: logprob-based, strategy-based, and LLM-based attacks, with other classes (shuffle, multi-round, flaw, and template-based methods) appearing less frequently. Specifically, Logprob-based attacks thrive because exposed diagnostics give highly informative feedback, making search efficient. Strategy-based attacks succeed since they exploit universal tendencies of LLMs to be helpful and follow instructions. LLM-based attacks dominate by automating generation and refinement, enabling scale and creativity with little manual effort. Overall, these three approaches dominate because they combine effectiveness, generalizability, and ease of execution, making them more practical than alternatives.

② **Jailbreak threats in practical black-box attack settings.** From the access and granularity perspective, the majority of attacks primarily involve prompt-level manipulations that operate in black-box settings, relying solely on input-output access to the target model. These attacks are impactful because they require no insider access or gradients: an adversary only needs the public API or chat interface to probe, iterate, and exploit behavioral weaknesses, which makes them easy to scale and automate. In addition, such attacks operate at the prompt level in a more computation-friendly manner than token-level approaches, i.e., they need fewer, coarser queries and less compute per candidate, lowering cost and latency of the attack, and thus posing a broad, low-barrier threat in practice.

③ **Red-teaming challenges using existing jailbreaks.** Despite their prevalence, most jailbreak attacks still rely heavily on human-guided strategies, which often produce low-diversity prompts that recycle fixed patterns and repetitive structures, making them easier to detect and less robust. At the same time, more advanced approaches, such as those logprob-based attacks involving optimization or external LLM orchestration, face significant trade-offs in computation and API cost, limiting scalability and reproducibility. Together, these factors show that existing jailbreak techniques are not yet fully automated, struggle to generate diverse adversarial prompts, and remain expensive to scale, challenges that make comprehensive and systematic red-teaming of LLMs difficult.

2.4. Defense Overview

To systematically examine defense strategies against jailbreaks in large language models (LLMs), we propose a taxonomy organized by the stage of defense deployment, as summarized in Table 2. This framework provides a structured overview that facilitates comparison of defenses across the model life cycle, while also highlighting the practical trade-offs associated with each approach.

Defense taxonomy. We categorize defense methods into five types based on their deployment stage in the model pipeline:

- **Pre-filter defenses** work by screening user inputs before they reach the target model. The idea is to detect and block

potentially malicious or jailbreak-inducing prompts at the entry point, which is usually implemented with auxiliary classifiers or with manually crafted rules.

- **System prompt defenses** strengthen safety by embedding explicit guardrails into the model’s system prompt. This involves inserting safety instructions or constraints that steer the model’s behavior during inference.
- **Fine-tuning-based defenses** improve robustness by re-training or further aligning the model itself on curated data. Instead of filtering inputs or relying on prompts, the model’s parameters are updated so that it inherently resists malicious instructions and produces safer outputs.
- **Intra-process defenses** intervene during inference by modifying or monitoring the model’s decoding process. Techniques may include constraining token sampling, biasing logits toward safe outputs, or interrupting decoding when unsafe continuations are detected.
- **Post-filter defenses** operate after the model has generated its response, filtering, revising, or blocking unsafe outputs before they reach the user. This approach can use classifiers, rule-based filters, or rewriting mechanisms to detect and suppress problematic content.

Key observations. From Table 2, we summarize the following key observations of the current defense methods:

① **White-box fine-tuning-based defenses dominate.** Fine-tuning is one of the most prevalent strategies because it offers a fundamental and durable form of robustness: once a model has been fine-tuned on carefully curated safety data, it can resist a wide range of attacks without requiring ongoing external interventions. This makes fine-tuning attractive as a long-term solution, since it internalizes safety into the model parameters and incurs minimal inference-time cost. However, this white-box dependency introduces significant scalability challenges. That is, each model must undergo its own fine-tuning or alignment process, which demands substantial compute, expertise, and high-quality data. As a result, while fine-tuning is powerful and deeply integrated, its lack of portability and heavy upfront overhead limit its flexibility in multi-model or black-box settings.

② **High computational cost of defenses.** Many existing defense methods introduce substantial computational overhead, either during training or at inference time. Approaches based on fine-tuning demand significant upfront resources for retraining and alignment, raising development costs and limiting scalability. Meanwhile, other defenses that avoid fine-tuning often shift the burden to inference: requiring multiple forward passes or auxiliary model calls to safeguard against a single potentially harmful query. This can multiply inference costs, degrade latency, and complicate deployment in real-world systems where efficiency is critical. Ultimately, this highlights a central tension: while strong defenses are desirable, the cost-benefit trade-off between robust safety guarantees and efficient service remains unresolved.

③ **Defensive efforts lag far behind attacks.** As shown in Table 1, the number and diversity of defensive studies are considerably smaller than that of attack methods. This imbalance arises in part because attacks are inherently easier to devise: adversaries need only find a single exploitative path-

TABLE 2: Overview and comparison of defense methods against jailbreak attacks. **Code:** ✓ indicates open-source code is available; ✗ no open-source implementation. **Access:** ● black-box access to the target model; ○ white-box access. **Flexibility:** ✓ means the defense is model-agnostic and transferable across LLMs; ✗ model-specific defense. **Extra LLM:** 🗡 an LLM is required; ○ LLMs are not required; **Fine-tuning requirement:** ⚙ requires fine-tuning procedures, either aligning the target model itself or fine-tuning an alternative model for defense; ○ fine-tuning is not needed; **Cost:** Training indicates the cost of defense is at the fine-tuning time of the target model; Iterative indicates the cost is at the inference time of the target model, with multiple rounds of inference to secure against a potential harmful prompt; Single-pass indicates cost is at the inference time of the target model with only one round for defense.

Type	Defense	Reference	Year	Code	Access	Flexibility	Extra LLM	Fine-tuning	Cost
Pre-filter	Erase-and-check	[36]	2023	✓	●	✓	○	○	Iterative
	LlamaGuard	[29]	2023	✓	●	✓	🗡	⚙	Single-pass
	Perplexity Filter	[1]	2023	✗	●	✓	🗡	⚙	Single-pass
	Hidden State Guard	[105]	2024	✓	●	✓	🗡	⚙	Single-pass
System prompt	SelfRemider	[80]	2023	✓	●	✓	○	○	Single-pass
Fine-tune	Constitutional AI	[5]	2022	✗	○	✗	○	⚙	Training
	Safe-RLHF	[14]	2023	✓	○	✗	○	⚙	Training
	Deep Alignment	[62]	2024	✗	○	✗	○	⚙	Training
	Safety-Tuned LLaMAs	[6]	2024	✓	○	✗	○	⚙	Training
	Safe unlearning	[100]	2024	✓	○	✗	○	⚙	Training
	CircuitBreaker	[107]	2024	✓	○	✗	○	⚙	Training
	Scot	[88]	2025	✓	○	✗	○	⚙	Training
Intra-process	PAT	[53]	2024	✓	○	✗	○	⚙	Single-pass
	ABD	[21]	2024	✗	○	✗	○	⚙	Single-pass
	SafeDecoding	[81]	2024	✓	○	✓	🗡	⚙	Single-pass
	JBSHield	[97]	2025	✓	○	✗	○	⚙	Single-pass
Post-filter	Smoothllm	[68]	2023	✓	●	✓	○	○	Iterative
	Aligner	[31]	2024	✓	○	✓	🗡	⚙	Single-pass
	Self-evaluation	[8]	2024	✓	●	✓	○	○	Single-pass
	Autodefense	[95]	2024	✓	●	✓	🗡	○	Iterative

way, whereas defenders must anticipate and guard against a broad and evolving range of threats. The implications of this gap are significant. If attack research continues to outpace defenses, LLMs may remain persistently vulnerable, with new jailbreak techniques regularly undermining existing safeguards. This asymmetry highlights the need for sustained investment in proactive and generalizable defense strategies, as well as better benchmarks to track progress and close the gap between offensive and defensive innovation.

3. Evaluation Framework

3.1. Security Cube

In this section, we introduce Security Cube, a novel multi-dimensional framework (Figure 1) for evaluating LLM robustness and safety along three axes: attacker, defender, and judge. Unlike prior benchmarks that examine isolated aspects, Security Cube unifies 7 attack, 3 defense, and 3 judge metrics, enabling a systematic mapping of the jailbreak landscape. Crucially, it introduces three new metrics, attack stability, concentration index per attack (CIPA), and depth of disruption, providing the first formal measures of adversarial consistency, concentration, and cascading impact. These innovations fill critical gaps in existing evaluations, offering a more comprehensive and structured assessment than prior frameworks. Section 3.2 and Table 3 further contrasts Security Cube with existing benchmarks. **Attacker axis** evaluates a jailbreak attack across five dimensions, success, stability, transferability, disruption depth, and

overhead, offering a quantitative, multidimensional basis for comparison, detection, and mitigation.

- **Attack success rate (ASR)**, denoted by α , the fraction of adversarial prompts that bypass a model’s safety mechanisms. Given a harmful prompt P , a jailbreak prompt P_j is constructed and the model produces a response R . A binary evaluation function $\text{Judge}(R, P_j)$ assigns $S = 1$ if the attempt is successful (i.e., the model violates its safety policy) and $S = 0$ otherwise. Over N total attempts, ASR is defined as

$$\alpha = \frac{1}{N} \sum_{k=1}^N S_k,$$

where larger α indicates more effective jailbreaks.

- **Attack stability**, denoted by β , measures the consistency of an attack in bypassing a model’s safety mechanisms under similar conditions (e.g., fixed temperature, varying random seeds). For a given harmful prompt P , a specific attack generates N repeated trials, yielding an attack success rate α . Over K distinct harmful prompts, this produces success rates $\alpha_1, \dots, \alpha_K$. The attack stability is then defined as the standard deviation of these success rates:

$$\beta = \text{std}(\alpha_1, \dots, \alpha_K),$$

where lower values of β indicate higher stability, i.e., the jailbreak attack is more reliably effective across prompts.

- **Attack transferability**. This metric family assesses the transferability of jailbreak attacks, with three concrete metrics define as follows:

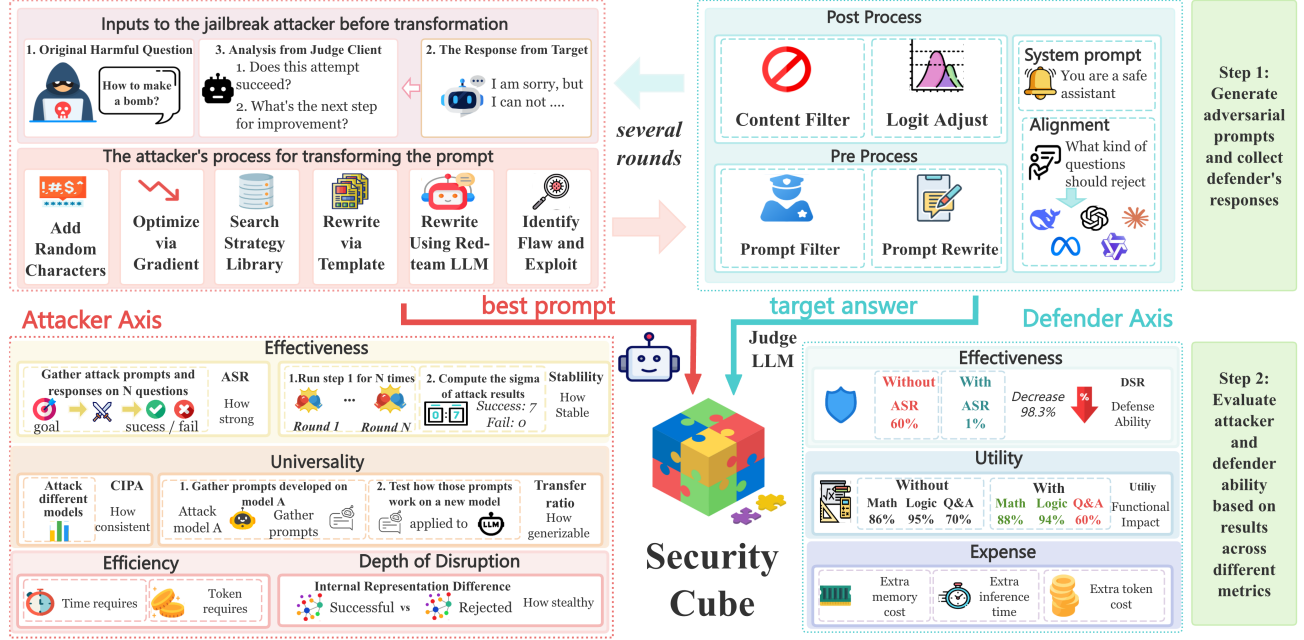


Figure 1: Overview of the Security Cube pipeline. Given a jailbreak goal, the attacker generates an initial adversarial prompt using a specific attack method (e.g., shuffling, LLM-based generation, or template rewriting). The target model, protected by a defense mechanism such as system prompts, pre-/post-guardrails, or other safety layers, produces a response. The attacker iteratively refines the prompt based on defender feedback (black-box or white-box), applying early stopping and incorporating suggestions. The final effective prompt–response pair is evaluated by a Judge model to assess attack success. Throughout the process, Security Cube logs key metrics of the attack, defense, and judge components.

- **Cross-model transferability**, denoted by η , quantifies how well an attack crafted for one model transfers to another model. Formally, consider N adversarial prompts P_i generated by a particular attack with respect to a source model. When these prompts are evaluated on a target model, let $S_i \in \{0, 1\}$ indicate whether the i -th prompt successfully bypasses the target model's safety mechanisms. The cross-model transferability is then defined as

$$\eta = \frac{1}{N} \sum_{i=1}^N S_i,$$

where higher values of η indicate that the attack generalizes more effectively across models.

- **Transfer ratio**, denoted by γ , measures the relative effectiveness of an attack on a target model compared to its effectiveness on the source model. Formally, let α denote the attack success rate on the source model and η the attack success rate on the target model (i.e., cross-model transferability). The transfer ratio is defined as

$$\gamma = \frac{\eta}{\alpha},$$

where values of γ close to 1 indicate that the attack transfers nearly as effectively as it works on the source model, values below 1 indicate weaker transfer to the

target, and values above 1 (though uncommon in practice) indicate that the attack is even more effective on the target than on the source.

- **Concentration Index per Attack (CIPA)**. Inspired by the Herfindahl–Hirschman Index [26], we define CIPA to quantify how concentrated or generalizable the success of an attack is across N different victim models. Let $\alpha_1, \dots, \alpha_N$ denote the attack success rates of the same attack on N models. Then CIPA is given by

$$\text{CIPA} = \sum_{i=1}^N \left(\frac{\alpha_i}{\sum_{j=1}^N \alpha_j} \right)^2,$$

where higher values indicate that the attack's success is concentrated on only a few models, while lower values indicate that the attack generalizes more broadly across different models.

Together, the three transferability metrics provide a multi-dimensional view of jailbreak robustness. Cross-model transferability and transfer ratio capture the portability of attacks, highlighting systemic risks when vulnerabilities easily transfer across models. CIPA reflects attack diversity, distinguishing between broad, generalizable jailbreaks and narrow, model-specific exploits. Particularly concerning are attacks with low CIPA and high transferability, as they are both widely effective and transferable,

whereas high CIPA with low transferability suggests more isolated and easier-to-mitigate weaknesses.

- **Depth of disruption**, denoted by μ , measures the representational divergence between successful and unsuccessful jailbreak prompts within a model. Let $\mathcal{D} = \{P_1, \dots, P_n\}$ be a dataset of prompts generated by a specific attack, partitioned into successful jailbreaks $\mathcal{D}_s$ and failures $\mathcal{D}_f$. Denote by $h(P)$ the hidden representation of prompt P at a given model layer. The depth of disruption at that layer is defined as

$$\mu = \text{dist}\left(\frac{1}{|\mathcal{D}_s|} \sum_{P \in \mathcal{D}_s} h(P), \frac{1}{|\mathcal{D}_f|} \sum_{P \in \mathcal{D}_f} h(P)\right),$$

where $\text{dist}(\cdot, \cdot)$ denotes a distance metric such as cosine distance. Higher values of μ indicate stronger representational divergence, reflecting more severe disruption of the model’s internal understanding caused by the attack.

- **Attack overhead** quantifies the computational cost of generating jailbreak prompts, measured in terms of both token consumption and elapsed time across N interaction rounds. The total token cost is defined as

$$T_{\text{token}} = \sum_{i=1}^N \left(T_{\text{prompt}}^{(i)} + T_{\text{output}}^{(i)} \right),$$

where $T_{\text{prompt}}^{(i)}$ and $T_{\text{output}}^{(i)}$ denote the number of input and output tokens, respectively, for round i . This includes tokens consumed by both the target model and any auxiliary red-teaming models employed during the attack. The total time cost is defined as

$$T_{\text{time}} = \sum_{i=1}^N T^{(i)},$$

where $T^{(i)}$ is the wall-clock time (in seconds) taken for the i -th interaction round. Higher values of attack overhead indicate greater resource demands, which may limit the practicality of a jailbreak in real-world settings.

Defender axis evaluates a jailbreak defense across three dimensions, defense effectiveness, overhead, and utility preservation, offering a balanced view of how well the defense mitigates attacks, its computational cost, and its impact on normal model functionality.

- **Defense success rate (DSR)**, denoted by ν , measures how effectively a defense reduces the success of jailbreak attacks, serving as the counterpart to ASR. It is defined as the relative reduction in attack success rate after applying a defense. Let α denote the original attack success rate on the undefended model, and let α_d denote the attack success rate after applying the defense. The defense success rate is defined as

$$\nu = \frac{\alpha - \alpha_d}{\alpha},$$

where $\nu = 0$ indicates that the defense has no effect, and $\nu = 1$ indicates that the defense completely prevents all successful attacks. Higher values of DSR therefore correspond to more effective defenses.

- **Utility preservation ability** measures the extent to which a defense maintains the model’s performance on standard tasks, thereby quantifying potential utility degradation. Let $\mathcal{D}$ be a dataset of clean (non-adversarial) tasks. The relative utility change is defined as

$$\Delta U = U^{\text{defense}}(\mathcal{D}) - U^{\text{undefended}}(\mathcal{D}),$$

where U^{defense} and $U^{\text{undefended}}$ denote task performance (e.g., accuracy) with and without the defense, respectively. Values of ΔU close to 0 indicate strong utility preservation, while negative values indicate degradation in the model’s normal utility due to the defense.

- **Defense overhead** quantifies the additional computational cost introduced by a defense mechanism, measured in terms of token usage, time, and memory. The token overhead is defined as

$$\Delta T_{\text{token}} = T_{\text{token}}^{\text{defense}} - T_{\text{token}}^{\text{undefended}},$$

where $T_{\text{token}}^{\text{defense}}$ and $T_{\text{token}}^{\text{undefended}}$ denote the total token consumption with and without the defense, respectively. Similarly, the time and memory overhead are defined as

$$\Delta T_{\text{time}} = T_{\text{time}}^{\text{defense}} - T_{\text{time}}^{\text{undefended}},$$

$$\Delta M = M^{\text{defense}} - M^{\text{undefended}}.$$

A defense is considered practical when ΔT_{token} , ΔT_{time} , and ΔM remain small, ensuring that security improvements do not come at excessive computational or economic cost.

Evaluator axis systematically evaluates automated judges by assessing their annotation quality against human experts and measuring their scalability in practice. Annotation quality is captured through agreement-based metrics (e.g., disagreement with humans, inter-annotator agreement), while evaluation cost quantifies the computational resources required for large-scale use.

- **Disagreement with humans** measures the reliability of an automated judge [11], [64], [83] by comparing its outputs against human annotations. Let α_h and α_j denote the attack success rates (ASRs) computed using human annotations and the automated judge, respectively. The disagreement is quantified as the absolute ASR bias:

$$\Delta \text{ASR} = |\alpha_h - \alpha_j|.$$

Lower values of ΔASR indicate closer alignment with human judgment (and thus higher reliability), while higher values reflect greater disagreement, suggesting the automated judge is less trustworthy.

- **Inter-annotator agreement** (κ) uses Fleiss’ kappa [20] to evaluate the consistency of annotations across human annotators and automated judges in labeling jailbreak attempts. Let N be the total number of jailbreak attempts, n the number of annotators, and $k = 2$ the number of categories (success or fail). Denote by n_{ij} the number of

TABLE 3: Comparison of our work with existing benchmarks, frameworks, and surveys on jailbreaks against LLMs. Each work is systematically compared across four key dimensions: attacks, defenses, judges, and evaluated target models. From the attacker perspective, we consider ASR, stability, transferability, disruption, and overhead. From the defender perspective, we assess DSR, utility preservation, and overhead. The judge perspective captures disagreement with humans, agreement consistency, and evaluation cost. Finally, the target model dimension covers variations in model size and version. ● and ○ represent cover and not cover, respectively.

Paper	Year	Type	Attacker Perspective					Defender Perspective			Judge Perspective			Target Model	
			ASR	Stability	Transferability	Disruption	Overhead	DSR	Utility-preservation	Overhead	Disagreement	Consistency	Cost	Size	Version
[19]	2023	Survey	●	○	○	○	○	○	○	○	○	○	○	○	○
[108]	2023	Benchmark	●	○	●	○	○	○	○	○	○	○	○	○	○
[83]	2024	Survey	●	○	○	○	○	○	○	○	○	○	○	○	○
[82]	2024	Benchmark	●	○	○	○	○	○	○	○	○	○	○	○	○
[7]	2024	Threat model	●	○	○	○	○	○	○	○	○	○	○	○	○
[51]	2024	Benchmark	●	○	○	○	○	○	○	○	○	○	○	○	○
[90]	2024	Survey	●	○	○	○	○	○	○	○	○	○	○	○	○
[89]	2024	Benchmark	●	○	○	○	○	○	○	○	○	○	○	○	○
[54]	2024	Benchmark	●	○	○	○	○	○	○	○	○	○	○	○	○
[35]	2024	Benchmark	●	○	○	○	○	○	○	○	○	○	○	○	○
[103]	2024	Framework	●	○	○	○	○	○	○	○	○	○	○	○	○
[56]	2024	Framework	●	○	○	○	○	○	○	○	○	○	○	○	○
[17]	2024	Framework	●	○	○	○	○	○	○	○	○	○	○	○	○
[75]	2024	Survey	●	○	○	○	○	○	○	○	○	○	○	○	○
[76]	2024	Benchmark	●	○	○	○	○	○	○	○	○	○	○	○	○
[74]	2024	Benchmark	●	○	○	○	○	○	○	○	○	○	○	○	○
[64]	2024	Framework	●	○	○	○	○	○	○	○	○	○	○	○	○
[11]	2024	Benchmark	●	○	○	○	○	○	○	○	○	○	○	○	○
[65]	2024	Survey	●	○	○	○	○	○	○	○	○	○	○	○	○
[92]	2024	Framework	●	○	○	○	○	○	○	○	○	○	○	○	○
[61]	2025	Survey	●	○	○	○	○	○	○	○	○	○	○	○	○
[99]	2025	Framework	●	○	○	○	○	○	○	○	○	○	○	○	○
[13]	2025	Benchmark	●	○	○	○	○	○	○	○	○	○	○	○	○
[71]	2025	Benchmark	●	○	○	○	○	○	○	○	○	○	○	○	○
[49]	2025	Survey	●	○	○	○	○	○	○	○	○	○	○	○	○
[106]	2025	Benchmark	●	○	○	○	○	○	○	○	○	○	○	○	○
[50]	2025	Benchmark	●	○	○	○	○	○	○	○	○	○	○	○	○
[9]	2025	Benchmark	●	○	○	○	○	○	○	○	○	○	○	○	○
Ours	2025	SoK	●	●	●	●	●	●	●	●	●	●	●	●	●

annotators assigning the i -th attempt to category j . The agreement for attempt i is

$$A_i = \frac{1}{n(n-1)} \sum_{j=1}^k n_{ij}(n_{ij} - 1),$$

and the mean observed agreement is

$$\bar{A} = \frac{1}{N} \sum_{i=1}^N A_i.$$

Let $a_j = \frac{1}{Nn} \sum_{i=1}^N n_{ij}$ denote the marginal probability of category j , and define the expected agreement as

$$\bar{A}_e = \sum_{j=1}^k a_j^2.$$

Fleiss' kappa is then given by

$$\kappa = \frac{\bar{A} - \bar{A}_e}{1 - \bar{A}_e},$$

which accounts for chance agreement. Higher values of κ indicate stronger consistency among annotators, with $\kappa = 1$ representing perfect agreement, $\kappa = 0$ chance-level agreement, and negative values indicating systematic disagreement among annotators.

- **Evaluation cost** quantifies the computational expense of using an automated judge, measured as the average token consumption per evaluation. Formally, for N evaluations, the cost is defined as

$$\tilde{T}_{\text{token}} = \frac{1}{N} \sum_{i=1}^N (T_i^{\text{prompt}} + T_i^{\text{output}}),$$

where T_i^{prompt} and T_i^{output} denote the input and output tokens consumed by the employed LLM(s) in the i -th evaluation. Lower values of $\tilde{T}_{\text{token}}$ indicate more cost-efficient automated judging.

3.2. Comparison to Existing Works

We present a systematic comparison of our work with 28 evaluation benchmarks, frameworks, and surveys on LLM jailbreaks in Table 3. The table highlights key characteristics across four dimensions: attacks, defenses, judges, and target models. From this comparison, we identify several major limitations of existing efforts as follows.

- ① **Limited metric coverage.** A major limitation in existing works is their over-reliance on a narrow set of metrics. Specifically, 20 out of 28 surveyed studies (over 70%) evaluate jailbreak attacks in LLMs solely through attack success rate. This singular focus limits understanding of how attacks generalize across diverse scenarios, and fails to capture the adaptive nature of real-world adversaries. Moreover, drawing conclusions based only on success rates risks misleading interpretations. For example, an attack that achieves a high success rate in one setting may not consistently succeed across different runs or contexts. Another issue is the widespread use of automated judges, such as AlpacaEval [41] and MT-Bench [4], to evaluate attacks and defenses. These frameworks typically restrict evaluation to a few coarse metrics (e.g., ASR or DSR). However, such metrics can be gamed. For instance, a trivial defense that always produces safe-looking but uninformative outputs can still achieve high scores and be deemed “robust” [101].
- ② **Limited attacks, defenses, and target model coverage.** Another major shortcoming of existing benchmarks lies

TABLE 4: Summary of LLMs used for jailbreak evaluation in Security Cube.

Model	Year	Size	Open / Closed Source	Access	Instr. Tuned	Alignment Method
GPT-3.5-Turbo [58]	2023	Not disclosed	Closed	API only	Yes	Not disclosed
Qwen-2.5-7B-Instruct [86]	2024	7B	Open (Apache-2.0)	Both	Yes	Online RL
LLaMA-3-8B-Instruct [23]	2024	8B	Open (llama3 license)	Both	Yes	Guardrails, Data Cleaning(pretrain), SFT, DPO [23]
Mistral-7B-Instruct-v0.2 [32]	2023	7B	Open (Apache-2.0)	Both	Yes	System Prompt [32]
o1-mini [30]	2024	Not disclosed	Closed	API only	Yes	Guardrails, Data Filtering(pretrain), Deliberative Alignment [30]
DeepSeek-v3 [43]	2024	631B	Open	Both	Yes	SafeRL [15]
Qwen-2.5-Max [86]	2025	Not disclosed	Closed	API only	Yes	SFT, RL [63]
Gemini-2.0-Flash [22]	2025	Not disclosed	Closed	API only	Yes	Guardrails, Data filtering(pretrain), SFT, RLHF [22]
Claude-3.7-Sonnet [3]	2025	Not disclosed	Closed	API only	Yes	Guardrails, RLHF [3]
Qwen3-235B-A22B [85]	2025	235B	Open (Apache-2.0)	Both	Yes	Not disclosed

in their limited scope across attacks, defenses, and target models. Many benchmarks cover only a narrow range of attack types and defenses, and evaluate on a small set of victim models. For example, some focus exclusively on prompt-based attacks [9], while others fail to test defenses across multiple model families or versions [11]. This limited coverage is particularly problematic given the rapid pace of LLM development, where models are continuously released with stronger alignment and safety mechanisms. As a result, an attack that was once considered effective may no longer succeed on newer versions, even within the same model family. Without broader evaluation, benchmarks risk presenting outdated or incomplete insights.

Overall, our proposed Security Cube addresses these gaps by broadening coverage across metrics, attacks, defenses, and target models, thereby capturing richer aspects of jailbreak dynamics. In doing so, it enables more systematic and reliable assessment of robustness in real-world, fast-evolving LLM ecosystems.

4. Security Cube Evaluation

4.1. Evaluation Setup

In this subsection, we outline the experimental setup of the selection of models, attacks, defenses, and judges evaluated in Security Cube. The selection is based on the principle to ensure coverage across all categories and provide a comprehensive view of the jailbreak landscape. Due to main page limits, the detailed description of models (Table 4), datasets, attack methods, defense methods, and judge is provided in Appendix A.

4.2. Model Robustness Landscape

We begin by assessing the robustness of a range of large language models (LLMs), summarized in Table 5, spanning releases from 2023 through 2025 and sizes from 7B to 671B parameters. Robustness is evaluated using several representative jailbreak attack methods. Table 5 reports the attack success rate (ASR) for each method-model pair, with warmer colors indicating higher ASRs.

Our analysis reveals a significant gap between earlier and more recent generations of LLMs. Legacy models released prior to 2025, such as GPT-3.5-Turbo, Qwen-2.5-7B-Instruct, and Mistral-7B-Instruct, show ASRs exceeding 90% under adaptive attacks, including LLM-Adaptive [2]

and GPTFuzzer [91], indicating substantial vulnerability. In contrast, the latest models, including Claude-3.7-Sonnet, Qwen-2.5-Max, and o1-mini, reduce ASRs under these strongest attacks to near zero, underscoring the impact of recent alignment advances. When aggregated across all attack methods, o1-mini and Claude-3.7-Sonnet deliver the lowest overall average ASR (approximately 17%), positioning them at the current frontier of robustness.

We hypothesize that these robustness improvements arise from a combination of mutually reinforcing factors: *i*) Deliberative alignment via chain-of-thought refusal mechanisms, enabling models to reason about safety constraints before generating answers and to internalize sophisticated refusal logic [24]; *ii*) Extensive external red-teaming and preparedness evaluations, where models undergo adversarial stress testing and iterative hardening [3], [30]; and *iii*) Defense-in-depth alignment frameworks, emphasizing safety-value learning, conflict resolution under ambiguous instructions, and reliability under uncertainty [59]. Collectively, these components constitute a qualitative leap in the robustness landscape, clearly differentiating post-2025 models from their predecessors.

Our primary evaluation leverages the well-established HarmBench dataset [51], released in early 2024. To address the possibility that robustness gains in newer models stem from exposure to HarmBench during training, we performed two complementary validation experiments, detailed in Appendix B. These results indicate that improvements cannot be attributed solely to dataset exposure, reinforcing the conclusion that recent alignment and safety mechanisms drive the observed gains.

Takeaway: Recent reasoning-aligned LLMs, such as o1-mini and Claude-3.7-Sonnet, show a qualitative leap in jailbreak robustness. Their resilience arises from deliberative refusal reasoning, large-scale red-teaming, and defense-in-depth alignment, marking a shift from reactive safeguards to integrated safety reasoning.

Open Problem: Yet, no model achieves full jailbreak resistance: advanced multi-turn and compositional attacks still yield >50 % ASR. Achieving generalizable robustness remains open, requiring scalable deliberative safety, mitigation of alignment-capability trade-offs, and theory-grounded robustness metrics beyond benchmarks.

TABLE 5: Attack success rate (ASR, in %) of different jailbreak methods across model families. Cooler shades (green) denote low ASR (0–37.5%, higher safety), yellow indicates moderate ASR (37.5–62.5%), and warmer shades (red) denote high ASR (62.5–100%, lower safety).

Attack Method	GPT-3.5	Qwen	Llama	Mistral	Qwen2.5-Max	DeepSeek	o1-mini	Claude-3-7	Gemini-2.0	Qwen3-235b
LLM-Adaptive	95.0	96.0	97.0	91.0	0.0	99.0	0.0	5.0	0.0	95.0
ActorBreaker	70.0	79.5	68.0	76.0	73.0	69.5	53.0	35.5	25.0	64.5
BON	42.5	20.0	11.0	23.0	26.5	38.0	3.5	4.5	4.0	10.0
Flip	62.0	49.0	19.5	26.0	51.0	80.5	0.0	26.5	27.5	73.5
PAIR	67.5	51.0	37.0	79.5	47.5	54.0	16.5	14.0	54.5	80.0
AutoDAN-Turbo	40.0	52.0	40.0	62.0	31.5	55.0	19.5	18.5	11.5	72.0
ReNeLLM	83.0	82.0	66.0	91.0	0.0	86.5	68.5	79.5	27.0	81.0
DrAttack	26.5	37.0	15.5	25.0	49.0	49.5	2.5	2.5	3.0	46.0
Multijail	68.5	35.0	47.0	48.0	18.0	21.0	2.0	17.0	13.0	28.0
CipherChat	9.5	14.0	20.5	68.0	13.0	40.0	0.5	42.5	15.5	50.0
CodeAttacker	49.5	58.0	57.5	73.0	69.5	56.0	33.0	40.0	28.0	80.0
PAP	62.5	75.0	39.0	60.0	61.5	63.5	10.0	15.0	33.5	66.0
GPTFuzzer	92.5	87.5	23.0	85.0	15.5	88.5	9.0	1.0	64.0	56.0
Avg	59.2	56.6	41.6	61.9	35.1	61.6	16.8	23.2	23.6	61.8

Model Names: **GPT-3.5**: GPT-3.5-Turbo; **Llama**: Meta-Llama-3-8B-Instruct; **Qwen**: Qwen2.5-7B-Instruct; **Mistral**: Mistral-7B-Instruct-v0.2; **DeepSeek**: DeepSeek-v3; **Qwen2.5-Max**: Qwen2.5-Max-2025-01-25; **o1-mini**: o1-mini; **Claude-3-7**: Claude-3-7-Sonnet-20250219; **Gemini-2.0**: Gemini-2.0-Flash; **Qwen3-235b**: Qwen3-235b.

4.3. Jailbreak Attack Comparisons

Attack concentration. Table 6 summarizes the average ASR and CIPA values for each attack. Specifically, the average ASR of an attack is obtained by averaging its ASRs across all models reported in Table 5, while the CIPA values are computed according to the metric defined in Section 3.1. Overall, ReNeLLM, ActorBreaker, and LLM-Adaptive emerge as the most effective strategies, achieving average success rates of 66.60%, 61.35%, and 57.65%, respectively. Meanwhile, CodeAttacker (CIPA = 0.11), ActorBreaker (0.11) exhibit the lowest CIPA values, indicating that these attacks generalize well across different models and thus possess broad applicability.

Notably, the most effective jailbreaks, those with both high average ASR and low CIPA, primarily fall into two categories: strategy-based and multi-round attacks, each offering distinct advantages. Strategy-based attacks leverage human insight to exploit model-specific vulnerabilities. Many are logic-driven, guiding the model to rationalize harmful outputs through carefully constructed reasoning chains. This design makes them robust across architectures and alignment settings. However, their reliance on human creativity limits scalability. Multi-round attacks are also effective because most alignment datasets primarily contain short, single-turn interactions, providing limited coverage of complex dialogue structures. Moreover, long-context exchanges may introduce distributional shifts [67], where dialogue patterns deviate from alignment data, reducing the reliability of safety mechanisms and enabling unsafe responses in later turns.

Cross-model attack transferability. To evaluate cross-model transferability, we construct harmful prompts on a source model for each attack and then evaluate them on a target model. The source models include DeepSeek-v3, Claude-3.7, Qwen2.5-7B, and LLaMA-3-8B, while the target models are Gemma-3-27B-it, Qwen3-8B, and LLaMA-3.2-3B. As shown in Table 7, the LLM-Adaptive attack achieves the highest average transfer success rate ($\approx 43\%$), indicating that exploiting consistent language-modeling behaviors, such as continuation from specific initial tokens (e.g., “sure”), can effectively induce jailbreaks across ar-

TABLE 6: Average ASR and CIPA scores for each attack method. For ASR, warmer red shades indicate higher values ($>55\%$), reflecting stronger attack performance. For CIPA, cooler green shades indicate lower values (<0.12), corresponding to greater attack concentration.

Attack Category	Attack Method	Average ASR $\uparrow$	CIPA (λ) $\downarrow$
Strategy	ReNeLLM	66.60	0.12
Multi-round	ActorBreaker	61.35	0.11
Logprob	LLM-Adaptive	57.65	0.16
Strategy	CodeAttack	54.57	0.11
Template	GPTFuzzer	52.24	0.14
LLM	PAIR	50.19	0.12
Strategy	PAP	48.58	0.12
Shuffle	Flip	41.52	0.13
LLM	AutoDAN-Turbo	40.17	0.12
Flaw	Multijail	29.74	0.14
Flaw	CipherChat	27.42	0.16
Strategy	DrAttack	25.63	0.15
Shuffle	BON	18.13	0.16

chitectures. PAP and ReNeLLM also demonstrate strong transferability, suggesting that they encode model-agnostic prompting strategies. Both employ task-diverting mechanisms that guide the model to perform an ostensibly benign task (e.g., code generation) while covertly producing harmful content. Since such “unaligned” regions in the latent space are shared among models, these attacks generalize effectively beyond their source architectures.

We further observe that several attacks (e.g., PAIR, Flip, and DrAttack) exhibit an average $\gamma > 1$, meaning their success rate on the target model exceeds that on the source model. This phenomenon likely arises when the source model used to craft harmful prompts is more capable or safety-aware than the target. Prompts generated under stronger alignment constraints may remain unsuccessful on the source model but can easily bypass weaker safety mechanisms in less aligned targets. These findings highlight an asymmetry in LLM jailbreak defenses: once advanced models become accessible to adversaries, weaker models without comparable alignment safeguards face elevated transfer risks.

Attack overhead. Table 8 reports the average token and time costs of each attack across multiple attack goals on Meta-Llama-3-8B-Instruct. Overall, token costs vary substantially due to differences in attack mechanisms. LLM-

TABLE 7: Average transfer ASR (%) and transfer ratio (γ) for each attack. For average transfer ASR, warmer red shades indicate higher values ($>30\%$), reflecting stronger attack transferability. For average γ , cooler green shades indicate lower values (<0.45), corresponding to weaker transferability.

Attack Method	Average Transfer ASR	Average γ
LLM-Adaptive	43.42	0.46
PAP	35.83	0.90
ReNeLLM	32.25	0.43
ActorBreaker	30.59	0.62
AutoDAN-Turbo	30.00	0.65
PAIR	28.37	1.34
Flip	22.92	1.32
Multijail	19.15	0.95
CodeAttacker	18.04	0.42
DrAttack	11.42	2.62
GPTFuzzer	7.84	0.31
BON	4.08	0.42
CipherChat	3.73	0.14

TABLE 8: Attack overhead for different methods on Meta-Llama-3-8B-Instruct, measured in token cost and time cost. Cooler green shades denote lower token and time costs (more computationally efficient attacks); warmer shades denote higher costs.

	Token Cost ↓	Time Cost(s) ↓
CodeAttacker	888.6	14.73
Multijail	1827.15	105.55
Flip	2405.5	9.73
CipherChat	3890.4	29.72
ReNeLLM	5682.1	48.13
DrAttack	13733.9	283.31
PAP	13789.5	47.81
GPTFuzzer	15959.2	121.46
BON	28780.6	127.43
AutoDAN-Turbo	57205.0	427.54
PAIR	74161.6	112.48
ActorBreaker	81789.1	335.71
LLM-Adaptive	444005.0	667.58

Adaptive incurs the highest cost because its gradient-based, white-box optimization process requires extensive token sampling, often hundreds of thousands of tokens, to refine prompts. In contrast, Flip and CodeAttacker, which are static and non-generative (involving only template-based formatting), incur negligible low costs, making them suitable for stealthy or low-resource attack scenarios. A mid-cost group, including AutoDAN-Turbo and PAIR, exhibits variable expenses depending on interaction depth. Notably, among these, AutoDAN-Turbo achieves higher efficiency, likely due to its strategy library or other prompt optimization mechanisms, resulting in reduced token consumption.

Attack stability. To assess attack stability, we select six common jailbreak goals, each represented by two questions. For every question, ten harmful prompts are generated using the evaluated attack method. As shown in Table 9, LLM-Adaptive exhibits the highest stability, with an average β of 0.03, attributed to its white-box, deterministic prompt generation process. This method consistently produces outputs initiated by fixed tokens (e.g., “Sure”), which minimizes contextual variance. CodeAttacker also demonstrates strong stability due to its template-based design and limited

prompt randomness. In contrast, ReNeLLM ($\beta = 0.44$) and PAIR ($\beta = 0.31$), although seeming to be effective with high averaged ASR in Table 6, show the lowest stability, with performance highly sensitive to contextual changes. This instability arises because both rely on LLM-generated prompts with weak structural constraints and unpredictable search paths, PAIR modifies prompts randomly without clear direction (e.g., identifying the model’s weaknesses), while ReNeLLM relies on the model to handle diverse tasks, but frequent prompt changes hinder its ability to perform consistently. Other attack methods maintain moderate stability, performing more consistently when confined to specific harmful goals.

Attack depth. To understand how attacks affect model internals, we compare the cosine similarity of hidden states between successful and failed prompts for each attack method. As shown in Figure 4 (Appendix C), most attacks exhibit their strongest impact in the final layers, where model decision-making occurs. For example, BON, PAIR, and CipherChat induce sharp shifts near the output, suggesting direct disruption of high-level semantic representations. Others, such as multijail and GPTFuzzer, show a gradual deviation from mid to deep layers, reflecting progressive reasoning manipulation. Figures 5a and 5b further illustrate how adversarial prompts alter the model’s internal representations. In Figure 5a (early layer), successful prompts from different attack methods form distinct clusters, indicating that each attack induces a consistent activation pattern, suggesting the feasibility of automated detection via hidden states. In Figure 5b (deep layer), benign and jailbreak prompts are clearly separated: benign inputs cluster tightly, while all jailbreak samples shift to a distinct region. This sharp divergence highlights fundamentally different representations, which could inform future defense designs.

Takeaway: *i) Attack effectiveness and generality* increasingly converge: strategy-based and multi-round approaches such as ReNeLLM, ActorBreaker, and LLM-Adaptive achieve both high average ASR and low CIPA, revealing that human-crafted reasoning strategies and dialogue-structure exploitation remain the most transferable jailbreak paradigms; *ii) Attack transferability* further exposes shared latent failure modes, linguistic continuation patterns and benign-task diversions, that persist across architectures, reflecting common generative priors rather than model-specific flaws; *iii) Attack resource cost and stability* vary widely: white-box adaptive attacks are computationally intensive but deterministic, whereas lightweight template-based methods offer low-cost, high-stability avenues for stealthy or automated exploitation.

Open Problem: *i) Unified optimization under conflicting attack objectives.* No existing jailbreak method simultaneously achieves high effectiveness, transferability, efficiency, and stability. Gains in one dimension often degrade another, exposing a lack of unified theory for multi-objective attack design. Understanding these trade-offs is

TABLE 9: Attack stability scores (β) for each attack method. Here, C denotes jailbreak categories, and Q_1 and Q_2 represent different questions within each category. Categories: C_1 chemical/biological; C_2 illegal; C_3 misinformation/disinformation; C_4 harmful; C_5 harassment/bullying; C_6 cybercrime/intrusion. Higher β values indicate lower attack stability. Color scale: red cells ($\beta > 0.45$), yellow cells ($0.2 < \beta \leq 0.45$), and green cells ($\beta \leq 0.2$).

Attack Method	(C_1, Q_1)	(C_1, Q_2)	(C_2, Q_1)	(C_2, Q_2)	(C_3, Q_1)	(C_3, Q_2)	(C_4, Q_1)	(C_4, Q_2)	(C_5, Q_1)	(C_5, Q_2)	(C_6, Q_1)	(C_6, Q_2)	Average
LLM-Adaptive	0.00	0.00	0.00	0.00	0.00	0.40	0.00	0.00	0.00	0.00	0.00	0.00	0.03
ActorBreaker	0.00	0.00	0.00	0.00	0.49	0.00	0.00	0.49	0.30	0.50	0.00	0.00	0.15
BON	0.00	0.50	0.00	0.42	0.00	0.31	0.00	0.00	0.42	0.00	0.00	0.31	0.16
Flip	0.40	0.00	0.00	0.49	0.00	0.00	0.00	0.00	0.40	0.00	0.00	0.00	0.11
PAIR	0.40	0.00	0.50	0.40	0.30	0.00	0.40	0.30	0.46	0.49	0.46	0.00	0.31
AutoDAN-Turbo	0.00	0.47	0.47	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.50	0.00	0.12
ReNeLLM	0.46	0.40	0.49	0.49	0.46	0.30	0.50	0.49	0.46	0.40	0.49	0.30	0.44
DrAttack	0.40	0.50	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.49	0.00	0.00	0.12
Multijail	0.49	0.00	0.00	0.30	0.00	0.00	0.50	0.46	0.40	0.00	0.00	0.00	0.18
CipherChat	0.46	0.00	0.49	0.50	0.30	0.00	0.00	0.49	0.30	0.40	0.30	0.00	0.27
CodeAttacker	0.00	0.40	0.00	0.00	0.00	0.00	0.00	0.30	0.00	0.00	0.00	0.00	0.06
PAP	0.46	0.30	0.00	0.00	0.00	0.00	0.49	0.00	0.00	0.40	0.40	0.30	0.20
GPTFuzzer	0.30	0.40	0.00	0.49	0.46	0.30	0.00	0.00	0.50	0.46	0.00	0.00	0.24

essential to define the limits of optimal jailbreak capability; *ii*) **Inherent vulnerability from shared generative priors.** LLMs remain intrinsically susceptible to jailbreaks due to their cooperation-oriented generative priors and autoregressive continuation bias. These shared linguistic and reasoning tendencies form systemic vulnerabilities that transcend architectures. Overcoming them demands rethinking safety at the level of generative modeling itself, not just post-hoc alignment.

4.4. Defense Method Comparisons

Defense success rate. Table 10 summarizes the defense success rate of each method. Hidden State Guard is the most effective, reducing attack success to nearly zero for nine out of eleven attacks. Its high performance arises from early-stage interception of malicious prompts by analyzing the model’s internal hidden representations, enabling accurate detection and blocking before generation. System Prompt SelfReminders also substantially lower Llama’s average ASR by reinforcing safety instructions through the system prompt, though their effect on Mistral is weaker, likely because Mistral’s inherent safety alignment is less robust, limiting the prompt’s influence. Aligner (a post-filter approach) offers partial protection but are less effective against sophisticated or multi-turn attacks. CircuitBreaker (a fine-tuning-based approach) may not reduce the attack success rate (ASR) to zero but significantly lowers it for most attack methods, providing overall protection. Overall, pre-filter-based defenses, particularly those leveraging internal model states, provide the most reliable protection. In contrast, system-prompt methods depend heavily on the model’s intrinsic alignment, and post-rewriting approaches tend to be less consistent in mitigating sophisticated jailbreaks.

Defense overhead. Table 11 compares the token, memory, and latency overhead across defenses. The fine-tuned CircuitBreaker incurs virtually no additional cost, as it leverages pre-existing model behavior without extra processing. SelfReminder introduces negligible overhead (<50 tokens per round) through lightweight prompt edits, with minimal latency as the input length increases slightly but memory remains unchanged. Hidden State Guard and LlamaGuard add a single extra model pass for safety checks, yielding minor

TABLE 10: Defense success rates (%) across different defenses. Color scale: red for high success ($>70\%$), yellow for moderate ($30\text{--}70\%$), and green for low ($<30\%$). Higher values indicate stronger defense performance.

Attack	Model	Defense Method				
		SelfReminder	LlamaGuard	Hidden State Guard	Aligner	CircuitBreaker
LLM-Adaptive	Mistral	16.13	99.44	100.00	97.76	89.93
	Llama	96.39	100.0	100.0	98.97	89.69
ActorBreaker	Mistral	15.23	21.19	31.79	64.90	62.91
	Llama	62.16	51.57	45.51	1.62	24.32
BON	Mistral	1.33	97.83	100.0	47.25	94.20
	Llama	100.0	100.0	100.0	72.73	90.91
Flip	Mistral	0.0	96.09	100.0	39.35	70.65
	Llama	89.74	94.87	100.0	5.13	76.92
PAIR	Mistral	15.09	24.53	96.23	11.95	46.54
	Llama	78.38	0.0	97.3	14.86	76.44
AutoDAN-Turbo	Mistral	52.98	31.48	99.43	24.50	82.85
	Llama	98.75	0.0	100.0	61.25	90.0
ReNeLLM	Mistral	37.36	59.34	99.45	31.86	68.68
	Llama	78.79	27.27	100.0	58.33	60.61
DrAttack	Mistral	0.0	56.07	100.0	32.88	23.02
	Llama	100.0	67.42	100.0	8.77	96.76
Multijail	Mistral	77.98	87.42	20.32	33.62	86.27
	Llama	94.68	72.34	48.94	51.15	78.07
CipherChat	Mistral	55.07	89.71	100.0	57.14	59.56
	Llama	100.0	21.95	100.0	0.0	100.0
CodeAttacker	Mistral	60.96	95.83	100.0	58.22	80.14
	Llama	93.04	93.04	100.0	31.30	28.7
PAP	Mistral	1.25	33.40	100.0	1.67	67.56
	Llama	93.59	35.90	100.0	33.33	96.15
GPTFuzzer	Mistral	70.27	98.25	98.83	84.73	97.03
	Llama	93.48	95.65	100.0	71.74	86.96

TABLE 11: Defense overhead of different defenses.

Overhead	CircuitBreaker	SelfReminder	Hidden State Guard	LlamaGuard	Aligner
Token	0	87.01	298.33	635.69	2278.53
Memory (MB)	0	0	5904.22	15316.51	5904.23
Latency (s)	0.21	1.36	3.23	0.82	29.68

resource increases; the former is more memory-efficient due to its smaller 7B model but incurs extra latency from an additional classifier pass, whereas the latter performs the check directly. In contrast, Aligner is the most resource-intensive, often exceeding 4K tokens due to its multi-stage rewriting process, which requires multiple output generations. Despite using a 7B model, its runtime remains slower, likely reflecting inefficiencies in model-structure optimization.

Defense-utility trade-off. As shown in Figure 3, most defenses have minimal impact on math and logic tasks. Hidden State Guard, SelfReminder, and LlamaGuard cause little to no accuracy drop on benign prompts, as the first two only filter prompts that pass, meaning the utility remains unaffected, and the system prompt does not harm the model’s performance if the question is benign. The fine-tuned CircuitBreaker exhibits a small accuracy drop

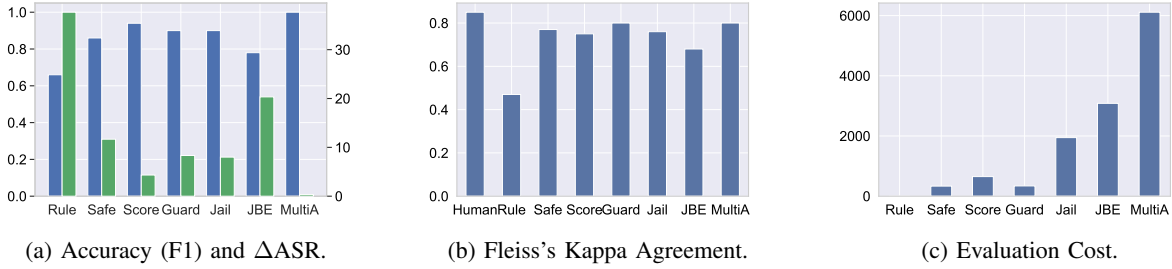


Figure 2: Comparison of Judge methods: Rule-based judge (Rule), binary Prompt-based (Binary Judge) (Safe), Prompt-based (Score Judge) (Score), LlamaGuard (Guard), JailJudge (Jail), JailbreakEvaluation (JBE), and Multi-Agent judge (MultiA). (a) Accuracy and alignment with human labels: Blue bars represent F1 score (higher is better) and green bars represent delta ASR (lower is better). (b) Agreement among automated judges and humans: Higher values indicate better agreement. (c) Cost in tokens: Lower values are better.

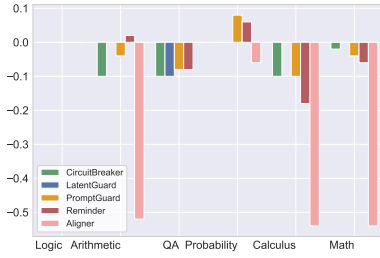


Figure 3: Utility change of each defense, measured as the percentage-point difference in task performance relative to the base model (Meta-Llama-3-8B-Instruct). Downward bars indicate reduced model utility after applying the defense. X-axis tasks: Logic, propositional logic; Arithmetic, arithmetic problems; QA, NFL question answering; Probability, probability tasks; Calculus, calculus problems; Math, graduate-level math tasks.

due to capability degradation from fine-tuning, but this has been largely mitigated, as the fine-tuning process preserves the model’s original ability across different tasks through careful loss function design. In contrast, Aligner induces performance declines due to answer rewriting, which can distort correct responses.

Takeaway: *i)* **Pre-filter works best:** Pre-filters based guards (e.g., Hidden State Guard and LlamaGuard) stop attacks early with minimal cost; *ii)* **System prompts are lightweight but limited:** SelfReminder adds little overhead but depends on strong base alignment; best used alongside fine-tuned safety; *iii)* **Post-generation fixes are inefficient:** Methods like Aligner are slow and distort outputs, while early detection is both safer and cheaper.

Open Problem: Combining complementary defenses, such as fine-tuning for stronger base alignment, followed by runtime filters like Hidden State Guard or SelfReminder to catch residual threats, could yield broader and more reliable protection. Evaluating such hybrid pipelines

presents a promising avenue for future research.

4.5. Judge Method Comparisons

Figure 2a shows that Multi-Agent judge align best with human judgments, achieving the lowest Δ ASR (0.34%) and highest F1 scores(0.99). Their multi-perspective design captures both surface-level and nuanced violations. Prompt-based (Score Judge) also outperforms prompt-based (Binary Judge: 11.66%). Fine-tuned models like LlamaGuard (8.33%) and JailJudge (8.00%) offer moderate alignment. Figure 2b shows that LlamaGuard and Multi-Agent yield the highest inter-annotator agreement ($\kappa = 0.80$), close to human level (0.84), indicating greater stability and reproducibility.

In cases where LLM judges diverge from human judgment, errors often arise from over-reliance on superficial lexical cues (e.g., “sure,” “here is a tutorial”) or misclassification of harmful content embedded in fictional contexts (e.g., movie). These weaknesses parallel behaviors exploited in red-teaming. In contrast, Multi-Agent and ensemble approaches mitigate such issues by integrating diverse evaluative perspectives, enabling cross-validation and more consistent judgments.

Figure 2c shows that LlamaGuard offers the best accuracy–cost trade-off, running 47 $\times$ faster and 18 $\times$ cheaper (in tokens) than Multi-Agent while maintaining strong performance (F1 = 0.90, $\kappa = 0.80$). The Multi-Agent Judge incurs higher cost due to its multi-round evaluation involving reasoning, guidance, instruction, answer, and goal assessments. In contrast, fine-tuned models like LlamaGuard require only input, answer, and goal evaluation, generating far fewer tokens. Prompt-based judges lie between these extremes, using limited guidance input but no iterative reasoning.

Takeaway: Multi-Agent achieves the highest alignment with human labels, albeit at a higher computational cost.

Open Problem: Future judge designs could adopt a multi-expert architecture, where specialized evaluators assess content from distinct dimensions, such as harm potential,

privacy risk, and ethical soundness, and a layered arbitration mechanism integrates these perspectives to enhance robustness, interpretability, and trustworthiness. However, it's crucial to explore the most effective sampling of perspectives and whether repeated cross-checking between agents is necessary, as without this, the cost of using multiple agents could become too high.

5. Outlook

Advancing LLM robustness through attack-defense co-evolution. Our analysis in experiments reveals that achieving jailbreak robust in LLMs requires moving beyond single-turn defenses and shallow alignment. First, future research should prioritize multi-turn and compositional robustness, as advanced attacks (e.g., ActorBreaker) exploit dialogue context and can bypass static guardrails. Second, representation-level defenses, such as monitoring hidden states or using circuit-breaker mechanisms, show promise for early detection and intervention, but need to be evaluated against adaptive, representation-aware attacks. Third, the research community would benefit from automated, evolving red-teaming pipelines and richer benchmarks (e.g., Jailbreak-Bench, HarmBench) that track not only ASR but also stability, transferability, and resource cost. Fourth, reasoning-aligned safety, where models are trained to deliberate about safety, not just follow rules, emerges as a key direction, especially as models become more agentic and tool-using. Finally, as jailbreak research outpaces defenses, the community should invest in living benchmarks, coordinated disclosure, and defense-in-depth strategies that combine fine-tuning, runtime filters, and post-deployment monitoring. Together, these directions aim to close the gap between attack and defense, and advance LLM safety from ad hoc mitigation toward principled, generalizable robustness.

Jailbreak interpretation. The observed effectiveness of hidden-state-based defenses suggests that adversarial prompts induce systematic and localized activation shifts in the underlying representation space, particularly within layers responsible for alignment and semantic abstraction. Investigating these activation dynamics could enable a deeper mechanistic understanding of how jailbreaks exploit model reasoning pathways. Future work should therefore aim to characterize and formalize the “trigger” patterns and representational signatures associated with alignment failures, integrating causal interpretability methods to trace how these internal perturbations propagate to behavioral outputs. Such an approach would bridge the gap between empirical defense design and theoretical insight, ultimately contributing to the development of models whose safety is grounded in transparent rather than post hoc output control.

Mode internals for advanced defense. A promising future direction lies in developing trajectory-aware defenses that reason about how prompts shape a model’s internal representations over the forward pass. Our analysis in Figure 4 and 5 shows that jailbreaks consistently induce structured deviations in hidden-state trajectories, either abrupt shifts

in the final layers or gradual drifts through mid to deep layers, revealing a geometric footprint of adversarial manipulation. Future defenses could therefore move beyond output filtering to monitor and interpret these representational trajectories, distinguishing benign from adversarial dynamics in real time. Such approaches would bridge interpretability and safety, enabling models that not only detect anomalous internal behavior but can also adaptively stabilize or correct it before harmful outputs emerge. More broadly, this line of work points toward a paradigm where robustness is enforced through representational governance, aligning model internals with safe reasoning processes rather than solely constraining surface behavior.

Evolving safety ecosystems. The development of adaptive, continuously evolving safety ecosystems for LLMs is essential. Rather than treating red-teaming and evaluation as static, one-off efforts, future research should pay more attention to explore automated, self-improving pipelines in which models are continually challenged by evolving adversarial strategies, e.g., “attacker AI agent”, and retrained to strengthen alignment over time. This vision draws on ideas from adversarial self-play and automated robustness testing, offering a path toward systems that can dynamically learn from their own failures. Complementing this, the field would benefit from community-maintained benchmarks and shared knowledge infrastructures that evolve alongside emerging threats. Establishing an open, regularly updated repository of jailbreak attacks, defenses, and evaluation protocols, i.e., developed collaboratively across research groups and organizations, would provide a living benchmark for progress, preventing stagnation and fostering reproducibility.

6. Conclusion

This paper systematizes jailbreak attacks and defenses on large language models (LLMs) through novel taxonomies grounded in their dominant underlying mechanisms. These taxonomies reveal structural commonalities across methods and enable a comprehensive synthesis of the evolving jailbreak landscape. Building on this foundation, we critically review existing benchmarks and evaluation frameworks, identifying their limitations in scope and metric coverage. To address these gaps, we introduce the Security Cube, a unified, multi-dimensional evaluation paradigm that extends beyond attack success rate to jointly characterize attacks, defenses, and judges. Using this framework, we evaluate 13 representative attacks, 5 defenses, and 4 judges under 7 attack, 3 defense, and 4 judge metrics, distilling key takeaways and open challenges.

Our analysis highlights fundamental tensions between capability, safety, and robustness, and points toward promising research directions in reasoning-aligned safety, interpretable jailbreak modeling, adaptive and efficient defenses, and a system-level safety ecosystem. Together, these directions aim to advance LLM security from ad hoc mitigation toward principled, generalizable robustness.

Acknowledgments

We thank all the reviewers for their insightful comments. This work was partially supported by the National Natural Science Foundation of China under grants No. 62532016, No. 6257071391, and No. 62502309, the Shanghai Municipal Education Commission under grant No. ZXDF030140, and the Natural Science Foundation of Shanghai under grant No. 25ZR1402211. Shuo Wang is the corresponding author.

Ethical Considerations

This work aims to strengthen LLM safety by systematically exposing vulnerabilities, not to promote misuse. All attacks were conducted in controlled environments, and no real-world harm was inflicted. We advocate for responsible red teaming and the open sharing of defense strategies to build safer AI systems.

References

- [1] Gabriel Alon and Michael Kamfonas. Detecting language model attacks with perplexity. *arXiv preprint arXiv:2308.14132*, 2023.
- [2] Maksym Andriushchenko, Francesco Croce, and Nicolas Flammarion. Jailbreaking leading safety-aligned llms with simple adaptive attacks. In *ICLR*, 2025.
- [3] Anthropic. Claude 3.7 sonnet system card. <https://www.anthropic.com/news/claude-3-7-sonnet>, 2025.
- [4] Ge Bai, Jie Liu, Xingyuan Bu, Yancheng He, Jiaheng Liu, Zhanhui Zhou, Zhuoran Lin, Wenbo Su, Tiezheng Ge, Bo Zheng, et al. Mt-bench-101: A fine-grained benchmark for evaluating large language models in multi-turn dialogues. In *ACL*, pages 7421–7454, 2024.
- [5] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- [6] Federico Bianchi, Mirac Suzgun, Giuseppe Attanasio, Paul Röttger, Dan Jurafsky, Tatsunori Hashimoto, and James Zou. Safety-tuned llamas: Lessons from improving the safety of large language models that follow instructions. In *ICLR*, 2024.
- [7] Valentyn Boreiko, Alexander Panfilov, Vaclav Voracek, Matthias Hein, and Jonas Geiping. A realistic threat model for large language model jailbreaks. *arXiv preprint arXiv:2410.16222*, 2024.
- [8] Hannah Brown, Leon Lin, Kenji Kawaguchi, and Michael Shieh. Self-evaluation as a defense against adversarial attacks on llms. *ArXiv*, abs/2407.03234, 2024.
- [9] Hongye Cao, Yanming Wang, Sijia Jing, Ziyue Peng, Zhixin Bai, Zhe Cao, Meng Fang, Fan Feng, Boyan Wang, Jiaheng Liu, Tianpei Yang, Jing Huo, Yang Gao, Fanyu Meng, Xi Yang, Chao Deng, and Junlan Feng. Safedialbench: A fine-grained safety benchmark for large language models in multi-turn dialogues with diverse jailbreak attacks. *ArXiv*, abs/2502.11090, 2025.
- [10] Marco Cascella, Jonathan Montomoli, Valentina Bellini, and Elena Bignami. Evaluating the feasibility of chatgpt in healthcare: an analysis of multiple clinical and research scenarios. *Journal of medical systems*, 47(1):33, 2023.
- [11] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwal, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. *NIPS*, 37:55005–55029, 2024.
- [12] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J Pappas, and Eric Wong. Jailbreaking black box large language models in twenty queries. In *SATML*, pages 23–42. IEEE, 2025.
- [13] Junjie Chu, Yugeng Liu, Ziqing Yang, Xinyue Shen, Michael Backes, and Yang Zhang. Comprehensive assessment of jailbreak attacks against llms. *arXiv preprint arXiv:2402.05668*, 2024.
- [14] Josef Dai, Xuehai Pan, Ruiyang Sun, Jiaming Ji, Xinbo Xu, Mickel Liu, Yizhou Wang, and Yaodong Yang. Safe rlhf: Safe reinforcement learning from human feedback. In *ICLR*, 2024.
- [15] DeepSeek-AI. Deepseek-rl: Incentivizing reasoning capability in llms via reinforcement learning, 2025.
- [16] Yue Deng, Wenxuan Zhang, Sinno Jialin Pan, and Lidong Bing. Multilingual jailbreak challenges in large language models. In *ICLR*, 2024.
- [17] Leon Derczynski, Erick Galinkin, Jeffrey Martin, Subho Majumdar, and Nanna Inie. garak: A framework for security probing large language models. *arXiv preprint arXiv:2406.11036*, 2024.
- [18] Peng Ding, Jun Kuang, Dan Ma, Xuezhi Cao, Yunsen Xian, Jiajun Chen, and Shujian Huang. A wolf in sheep’s clothing: Generalized nested jailbreak prompts can fool large language models easily. In *NAACL*, pages 2136–2153, 2024.
- [19] Aysan Esmradi, Daniel Wankit Yip, and Chun Fai Chan. A comprehensive survey of attack techniques, implementation, and mitigation strategies in large language models. In *UbiSec*, 2023.
- [20] Joseph L Fleiss. Measuring nominal scale agreement among many raters. *Psychological bulletin*, 76(5):378, 1971.
- [21] Lang Gao, Xiangliang Zhang, Preslav Nakov, and Xiuying Chen. Shaping the safety boundaries: Understanding and defending against jailbreaks in large language models. In *ACL*, 2025.
- [22] Google. Gemini-2.0-flash documents. <https://modelcards.withgoogle.com/assets/documents/gemini-2-flash.pdf>, 2025.
- [23] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [24] Melody Y. Guan, Manas Joglekar, Eric Wallace, Saachi Jain, Boaz Barak, Alec Helyar, Rachel Dias, Andrea Vallone, Hongyu Ren, Jason Wei, Hyung Won Chung, Sam Toyer, Johannes Heidecke, Alex Beutel, and Amelia Glaese. Deliberative alignment: Reasoning enables safer language models, 2025.
- [25] Xingang Guo, Fangxu Yu, Huan Zhang, Lianhui Qin, and Bin Hu. Cold-attack: Jailbreaking llms with stealthiness and controllability. In *ICML*, pages 16974–17002. PMLR, 2024.
- [26] Orris C Herfindahl. *Concentration in the steel industry*. Columbia university, 1997.
- [27] Xinzhe Huang, Kedong Xiu, Tianhang Zheng, Churui Zeng, Wangze Ni, Zhan Qiin, Kui Ren, and Chun Chen. Dualbreach: Efficient dual-jailbreaking via target-driven initialization and multi-target optimization. *ArXiv*, abs/2504.18564, 2025.
- [28] John Hughes, Sara Price, Aengus Lynch, Rylan Schaeffer, Fazl Barez, Sanmi Koyejo, Henry Sleight, Erik Jones, Ethan Perez, and Mrinank Sharma. Best-of-n jailbreaking. *arXiv preprint arXiv:2412.03556*, 2024.
- [29] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabza. Llama guard: Llm-based input-output safeguard for human-ai conversations, 2023.
- [30] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

- [31] Jiaming Ji, Boyuan Chen, Hantao Lou, Donghai Hong, Borong Zhang, Xuehai Pan, Tianyi Qiu, Juntao Dai, and Yaodong Yang. Aligner: Efficient alignment by learning to correct. In *NIPS*, 2024.
- [32] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7b, 2023.
- [33] Fengqing Jiang, Zhangchen Xu, Luyao Niu, Zhen Xiang, Bhaskar Ramasubramanian, Bo Li, and Radha Poovendran. Artprompt: Ascii art-based jailbreak attacks against aligned llms. In *ACL*, 2024.
- [34] Enkelejda Kasneci, Kathrin Se  ler, Stefan K  chemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan G  nnemann, Eyke H  llermeier, et al. Chatgpt for good? on opportunities and challenges of large language models for education. *Learning and individual differences*, 103:102274, 2023.
- [35] Moussa Koulako Bala Doumbouya, Ananjan Nandi, Gabriel Poesia, Davide Ghilardi, Anna Goldie, Federico Bianchi, Dan Jurafsky, and Christopher D Manning. h4rm3l: A dynamic benchmark of composable jailbreak attacks for llm safety assessment. *arXiv e-prints*, pages arXiv-2408, 2024.
- [36] Aounon Kumar, Chirag Agarwal, Suraj Srinivas, Aaron Jiaxun Li, Soheil Feizi, and Himabindu Lakkaraju. Certifying llm safety against adversarial prompting. In *COLM*, 2024.
- [37] Hongyi Li, Jiawei Ye, Jie Wu, Tianjie Yan, Chu Wang, and Zhixin Li. Jailpo: A novel black-box jailbreak framework via preference optimization against aligned llms. In *AAAI*, pages 27419-27427, 2025.
- [38] Linbao Li, Yunnan Liu, Daojing He, and Yu Li. One model transfer to all: On robust jailbreak prompts generation against llms. In *ICLR*, 2025.
- [39] Xirui Li, Ruochen Wang, Minhao Cheng, Tianyi Zhou, and Cho-Jui Hsieh. Drattack: Prompt decomposition and reconstruction makes powerful llm jailbreakers. In *EMNLP*, 2024.
- [40] Xuan Li, Zhanke Zhou, Jianing Zhu, Jiangchao Yao, Tongliang Liu, and Bo Han. Deepinception: Hypnotize large language model to be jailbreaker. *arXiv preprint arXiv:2311.03191*, 2023.
- [41] Xuechen Li, Tianyi Zhang, Yann Dubois, Rohan Taori, Ishaan Gulrajani, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. AlpacaEval: An automatic evaluator of instruction-following models. https://github.com/tatsu-lab/alpaca_eval, 5 2023.
- [42] Zeyi Liao and Huan Sun. Amplegcg: Learning a universal and transferable generative model of adversarial suffixes for jailbreaking both open and closed llms. In *COLM*, 2024.
- [43] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [44] Fan Liu, Yue Feng, Zhao Xu, Lixin Su, Xinyu Ma, Dawei Yin, and Hao Liu. Jailjudge: A comprehensive jailbreak judge benchmark with multi-agent enhanced explanation evaluation framework. *ArXiv*, abs/2410.12855, 2024.
- [45] Xiaogeng Liu, Peiran Li, G. Edward Suh, Yevgeniy Vorobeychik, Zhuoqing Mao, Somesh Jha, Patrick McDaniel, Huan Sun, Bo Li, and Chaowei Xiao. AutoDAN-turbo: A lifelong agent for strategy self-exploration to jailbreak LLMs. In *ICLR*, 2025.
- [46] Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. Autodan: Generating stealthy jailbreak prompts on aligned large language models. In *ICLR*, 2024.
- [47] Yanjiang Liu, Shuheng Zhou, Yaojie Lu, Huijia Zhu, Weiqiang Wang, Hongyu Lin, Ben He, Xianpei Han, and Le Sun. Auto-rt: Automatic jailbreak strategy exploration for red-teaming large language models. *ArXiv*, abs/2501.01830, 2025.
- [48] Yue Liu, Xiaoxin He, Miao Xiong, Jinlan Fu, Shumin Deng, and Bryan Hooi. Flipattack: Jailbreak llms via flipping. *arXiv preprint arXiv:2410.02832*, 2024.
- [49] Xingjun Ma, Yifeng Gao, Yixu Wang, Ruofan Wang, Xin Wang, Ye Sun, Yifan Ding, Hengyuan Xu, Yunhao Chen, Yunhan Zhao, et al. Safety at scale: A comprehensive survey of large model safety. *arXiv preprint arXiv:2502.05206*, 2025.
- [50] Wuyao Mai, Geng Hong, Pei Chen, Xudong Pan, Baojun Liu, Yuan Zhang, Haixin Duan, and Min Yang. You can't eat your cake and have it too: The performance degradation of llms with jailbreak defense. In *WWW*, pages 872-883, 2025.
- [51] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhae, Nathaniel Li, Steven Basart, Bo Li, et al. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. In *ICML*, pages 35181-35224. PMLR, 2024.
- [52] Anay Mehrotra, Manolis Zampetakis, Paul Kassianik, Blaine Nelson, Hyrum Anderson, Yaron Singer, and Amin Karbasi. Tree of attacks: Jailbreaking black-box llms automatically. *NIPS*, 37:61065-61105, 2024.
- [53] Yichuan Mo, Yuji Wang, Zeming Wei, and Yisen Wang. Fight back against jailbreaking via prompt adversarial tuning. In *NIPS*, 2024.
- [54] Yutao Mou, Shikun Zhang, and Wei Ye. Sg-bench: Evaluating llm safety generalization across diverse tasks and prompt types. *NIPS*, 37:123032-123054, 2024.
- [55] Zvi Mowshowitz., 2022. Accessed: 2024-02- 25.
- [56] Gary D Lopez Munoz, Amanda J Minnich, Roman Lutz, Richard Lundeen, Raja Sekhar Rao Dheekonda, Nina Chikanov, Bolor-Erdene Jagdagdorj, Martin Pouliot, Shiven Chawla, Whitney Maxwell, et al. Pyrit: A framework for security risk identification and red teaming in generative ai system. *arXiv preprint arXiv:2410.02828*, 2024.
- [57] Zhenxing Niu, Haodong Ren, Xinbo Gao, Gang Hua, and Rong Jin. Jailbreaking attack against multimodal large language model. *arXiv preprint arXiv:2402.02309*, 2024.
- [58] OpenAI. gpt-3.5-turbo system card. <https://platform.openai.com/docs/models/gpt-3.5-turbo>, 2023.
- [59] OpenAI. How we think about safety and alignment. <https://openai.com/safety/how-we-think-about-safety-alignment/>, 2025.
- [60] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *NIPS*, 35:27730-27744, 2022.
- [61] Benji Peng, Ziqian Bi, Qian Niu, Ming Liu, Pohsun Feng, Tianyang Wang, Lawrence KQ Yan, Yizhu Wen, Yichao Zhang, and Caitlyn Heqi Yin. Jailbreaking and mitigation of vulnerabilities in large language models. *arXiv preprint arXiv:2410.15236*, 2024.
- [62] Xiangyu Qi, Ashwinee Panda, Kaifeng Lyu, Xiao Ma, Subhrajit Roy, Ahmad Beirami, Prateek Mittal, and Peter Henderson. Safety alignment should be made more than just a few tokens deep. In *ICLR*, 2025.
- [63] Qwen. Qwen-2.5-max technical report. https://www.researchgate.net/publication/388485555_Technical_Report_on_Qwen-2.5_Max, 2025.
- [64] Delong Ran, Jinyuan Liu, Yichen Gong, Jingyi Zheng, Xinlei He, Tianshuo Cong, and Anyu Wang. JailbreakEval: An integrated toolkit for evaluating jailbreak attempts against large language models. *ArXiv*, abs/2406.09321, 2024.
- [65] Abhinav Rao, Sachin Vashistha, Atharva Naik, Somak Aditya, and Monojit Choudhury. Tricking llms into disobedience: Formalizing, analyzing, and detecting jailbreaks. In *LREC*, 2023.

- [66] Qibing Ren, Chang Gao, Jing Shao, Junchi Yan, Xin Tan, Wai Lam, and Lizhuang Ma. Codeattack: Revealing safety generalization challenges of large language models via code completion. In *ACL*, pages 11437–11452, 2024.
- [67] Qibing Ren, Hao Li, Dongrui Liu, Zhanxu Xie, Xiaoya Lu, Yu Qiao, Lei Sha, Junchi Yan, Lizhuang Ma, and Jing Shao. LLMs know their vulnerabilities: Uncover safety gaps through natural distribution shifts. *ACL*, 2025.
- [68] Alexander Robey, Eric Wong, Hamed Hassani, and George J. Papas. Smoothllm: Defending large language models against jailbreaking attacks. *ArXiv*, abs/2310.03684, 2023.
- [69] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- [70] Mark Russinovich, Ahmed Salem, and Ronen Eldan. Great, now write an article about that: The crescendo multi-turn llm jailbreak attack. In *USENIX Security*, pages 2421–2440, 2025.
- [71] Guobin Shen, Dongcheng Zhao, Linghao Feng, Xiang He, Jihang Wang, Sicheng Shen, Haibo Tong, Yiting Dong, Jindong Li, Xiang Zheng, et al. Pandaguard: Systematic evaluation of llm safety against jailbreaking attacks. *arXiv preprint arXiv:2505.13862*, 2025.
- [72] Xinyue Shen, Zeyuan Chen, Michael Backes, Yun Shen, and Yang Zhang. "do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models. In *ACM SIGSAC*, pages 1671–1685, 2024.
- [73] Chawin Sitawarin, Norman Mu, David Wagner, and Alexandre Araujo. Pal: Proxy-guided black-box attack on large language models. *ArXiv*, abs/2402.09674, 2024.
- [74] Lichao Sun, Yue Huang, Haoran Wang, Siyuan Wu, Qihui Zhang, Chujie Gao, Yixin Huang, Wenhan Lyu, Yixuan Zhang, Xiner Li, et al. Trustllm: Trustworthiness in large language models. In *ICML*, pages 20166–20270, 2024.
- [75] Sadaf Surur, J. Angel, and Arul Jothi. Jailbreak attacks on large language models and possible defenses: Present status and future possibilities. *ISTAS*, pages 1–7, 2024.
- [76] Boxin Wang, Weixin Chen, Hengzhi Pei, Chulin Xie, Mintong Kang, Chenhui Zhang, Chejian Xu, Zidi Xiong, Ritik Dutta, Rylan Schaeffer, et al. Decodingtrust: A comprehensive assessment of trustworthiness in gpt models. *NIPS*, 36:31232–31339, 2023.
- [77] Fengxiang Wang, Ranjie Duan, Peng Xiao, Xiaojun Jia, Shiji Zhao, Cheng Wei, YueFeng Chen, Chongwen Wang, Jialing Tao, Hang Su, et al. Mrj-agent: An effective jailbreak agent for multi-round dialogue. *arXiv preprint arXiv:2411.03814*, 2024.
- [78] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *EMNLP*, pages 8696–8708. Association for Computational Linguistics (ACL), 2021.
- [79] Jochen Wulf and Jürg Meierhofer. Exploring the potential of large language models for automation in technical customer service. *arXiv preprint arXiv:2405.09161*, 2024.
- [80] Yueqi Xie, Jingwei Yi, Jiawei Shao, Justin Curl, Lingjuan Lyu, Qifeng Chen, Xing Xie, and Fangzhao Wu. Defending chatgpt against jailbreak attack via self-reminders. *Nature Machine Intelligence*, 5:1486–1496, 2023.
- [81] Zhangchen Xu, Fengqing Jiang, Luyao Niu, Jinyuan Jia, Bill Yuchen Lin, and Radha Poovendran. Safedecoding: Defending against jailbreak attacks via safety-aware decoding. In *ACL*, pages 5587–5605, 2024.
- [82] Zhao Xu, Fan Liu, and Hao Liu. Bag of tricks: Benchmarking of jailbreak attacks on llms. *NIPS*, 37:32219–32250, 2024.
- [83] Zihao Xu, Yi Liu, Gelei Deng, Yuekang Li, and Stjepan Picek. A comprehensive study of jailbreak attack versus defense for large language models. In *ACL*, pages 7432–7449, 2024.
- [84] Yanni Xue, Jiakai Wang, Zixin Yin, Yuqing Ma, Haotong Qin, Renshuai Tao, and Xianglong Liu. Dual intention escape: Penetrating and toxic jailbreak attack against large language models. *WWW*, 2025.
- [85] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [86] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [87] Haoming Yang, Ke Ma, Xiaojun Jia, Yingfei Sun, Qianqian Xu, and Qingming Huang. Cannot see the forest for the trees: Invoking heuristics and biases to elicit irrational choices of llms. In *ICML*, 2025.
- [88] Xianglin Yang, Gelei Deng, Jieming Shi, Tianwei Zhang, and Jin Song Dong. Enhancing model defense against jailbreaks with proactive safety reasoning. *arXiv preprint arXiv:2501.19180*, 2025.
- [89] Yahan Yang, Soham Dan, Dan Roth, and Insup Lee. Benchmarking llm guardrails in handling multilingual toxicity. *ArXiv*, abs/2410.22153, 2024.
- [90] Sibo Yi, Yule Liu, Zhen Sun, Tianshuo Cong, Xinlei He, Jiaxing Song, Ke Xu, and Qi Li. Jailbreak attacks and defenses against large language models: A survey. *ArXiv*, abs/2407.04295, 2024.
- [91] Jiahao Yu, Xingwei Lin, Zheng Yu, and Xinyu Xing. Gptfuzzer: Red teaming large language models with auto-generated jailbreak prompts. *arXiv preprint arXiv:2309.10253*, 2023.
- [92] Zhiyuan Yu, Xiaogeng Liu, Shunning Liang, Zach Cameron, Chaowei Xiao, and Ning Zhang. Don't listen to me: Understanding and exploring jailbreak prompts of large language models. In *USENIX Security*, pages 4675–4692, 2024.
- [93] Youliang Yuan, Wenxiang Jiao, Wenxuan Wang, Jen-tse Huang, Pinjia He, Shuming Shi, and Zhaopeng Tu. Gpt-4 is too smart to be safe: Stealthy chat with llms via cipher. In *ICLR*, 2023.
- [94] Yi Zeng, Hongpeng Lin, Jingwen Zhang, Diyi Yang, Ruoxi Jia, and Weiyan Shi. How johnny can persuade llms to jailbreak them: Rethinking persuasion to challenge ai safety by humanizing llms. In *ACL*, pages 14322–14350, 2024.
- [95] Yifan Zeng, Yiran Wu, Xiao Zhang, Huazheng Wang, and Qingyun Wu. Autodefense: Multi-agent llm defense against jailbreak attacks. *ArXiv*, abs/2403.04783, 2024.
- [96] Qiang Zhang, Keyan Ding, Tianwen Lv, Xinda Wang, Qingyu Yin, Yiwen Zhang, Jing Yu, Yuhao Wang, Xiaotong Li, Zhuoyi Xiang, et al. Scientific large language models: A survey on biological & chemical domains. *ACM Computing Surveys*, 57(6):1–38, 2025.
- [97] Shenyi Zhang, Yuchen Zhai, Keyan Guo, Hongxin Hu, Shengnan Guo, Zheng Fang, Lingchen Zhao, Chao Shen, Cong Wang, and Qian Wang. Jbshield: defending large language models from jailbreak attacks through activated concept analysis and manipulation. In *USENIX Security*, 2025.
- [98] Yihao Zhang and Zeming Wei. Boosting jailbreak attack with momentum. In *ICASSP*, pages 1–5. IEEE, 2025.
- [99] Zhixin Zhang, Leqi Lei, Junxiao Yang, Xijie Huang, Yida Lu, Shiyao Cui, Renmiao Chen, Qinglin Zhang, Xinyuan Wang, Hao Wang, Hao Li, Xianqi Lei, Chengwei Pan, Lei Sha, Hongning Wang, and Minlie Huang. Aisafetyslab: A comprehensive framework for ai safety evaluation and improvement. *ArXiv*, abs/2502.16776, 2025.
- [100] Zhixin Zhang, Junxiao Yang, Yida Lu, Pei Ke, Shiyao Cui, Chujie Zheng, Hongning Wang, and Minlie Huang. From theft to bomb-making: The ripple effect of unlearning in defending against jailbreak attacks. *arXiv preprint arXiv:2407.02855*, 2024.
- [101] Xiaosen Zheng, Tianyu Pang, Chao Du, Qian Liu, Jing Jiang, and Min Lin. Cheating automatic llm benchmarks: Null models achieve high win rates. In *ICLR*, 2025.

- [102] Andy Zhou and Ron Arel. Tempest: Autonomous multi-turn jailbreaking of large language models with tree search. *arXiv preprint arXiv:2503.10619*, 2025.
- [103] Weikang Zhou, Xiao Wang, Limao Xiong, Han Xia, Yingshuang Gu, Mingxu Chai, Fukang Zhu, Caishuang Huang, Shihan Dou, Zhiheng Xi, et al. Easyjailbreak: A unified framework for jailbreaking large language models. *arXiv preprint arXiv:2403.12171*, 2024.
- [104] Yukai Zhou and Wenjie Wang. Don’t say no: Jailbreaking llm by suppressing refusal. In *ACL*, 2025.
- [105] Zhenhong Zhou, Haiyang Yu, Xinghua Zhang, Rongwu Xu, Fei Huang, and Yongbin Li. How alignment and jailbreak work: Explain llm safety through intermediate hidden states. In *EMNLP*, pages 2461–2488, 2024.
- [106] Giulio Zizzo, Giandomenico Cornacchia, Kieran Fraser, Muhammad Zaid Hameed, Ambrish Rawat, Beat Buesser, Mark Purcell, Pin-Yu Chen, Prasanna Sattigeri, and Kush Varshney. Adversarial prompt evaluation: Systematic benchmarking of guardrails against prompt input attacks on llms. *arXiv preprint arXiv:2502.15427*, 2025.
- [107] Andy Zou, Long Phan, Justin Wang, Derek Duenas, Maxwell Lin, Maksym Andriushchenko, J Zico Kolter, Matt Fredrikson, and Dan Hendrycks. Improving alignment and robustness with circuit breakers. *NIPS*, 37:83345–83373, 2024.
- [108] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models, 2023.
- [109] Qingsong Zou, Jingyu Xiao, Qing Li, Zhi Yan, Yuhang Wang, Li Xu, Wenxuan Wang, Kuofeng Gao, Ruoyu Li, and Yong Jiang. Query-attack: Jailbreaking aligned large language models using structured non-natural query language. In *ACL*, 2025.

Appendix A. Experimental Setting

In this section, we outline the experimental setup of the selection of models, attacks, defenses, and judges evaluated in Security Cube. The selection is based on the principle to ensure coverage across all categories and provide a comprehensive view of the jailbreak landscape.

Models. In Table 4, we present a comprehensive summary of the evaluated models, spanning both open and closed systems across different sizes, release years, access modalities, and alignment approaches. This selection captures the diversity of today’s LLM landscape, ensuring broad and representative coverage for our jailbreak evaluation. The temperature is set to 0.1 for all models.

Dataset. The adversarial goals in our experiments are drawn from the well-established HarmBench [51] benchmark, which contains 200 distinct harmful objectives. Across all evaluated models, we executed more than 48,000 attack attempts. In each attempt, a single attack method was applied to craft a harmful prompt, which was then presented to the target model either directly or the model is applied with a defense mechanism. This large-scale evaluation produced a rich dataset of model responses, forming the basis for our subsequent analysis of jailbreak effectiveness and defense performance. Additionally, we use Jailbreak-Bench [11] benchmark containing 300 questions annotated by 3 human experts to evaluate the performance of different judge methods.

TABLE 12: ASR of different model categories: older models (released before HarmBench) and newer models (released after HarmBench). ASR is reported on the constructed new dataset with minimal overlap with HarmBench. Cooler shades (green) denote low ASR (0–37.5%, higher safety), yellow indicates moderate ASR (37.5–62.5%), and warmer shades (red) denote high ASR (62.5–100%, lower safety).

Attack Method	Older Models				Newer Models				
	GPT-3.5	Llama	Qwen	Mistral	DeepSeek	Qwen2.5-Max	o1-mini	Claude-3-7	Qwen3-235b
llmadaptive	74.0	96.0	96.0	94.0	98.0	74.0	0.0	8.0	94.0
actorattack	68.0	78.0	84.0	82.0	80.0	80.0	76.0	50.0	82.0
bon	32.0	6.0	6.0	48.0	34.0	12.0	10.0	10.0	10.0
flip	34.0	18.0	58.0	22.0	82.0	38.0	2.0	14.0	70.0
pair	58.0	54.0	74.0	56.0	78.0	62.0	36.0	32.0	74.0
autodanturbo	60.0	66.0	86.0	88.0	86.0	76.0	66.0	18.0	72.0
renellm	56.0	44.0	78.0	86.0	72.0	66.0	48.0	36.0	84.0
drattack	32.0	14.0	20.0	54.0	50.0	58.0	20.0	24.0	54.0
multijail	44.0	30.0	20.0	24.0	10.0	6.0	6.0	2.0	14.0
cipherchat	14.0	16.0	32.0	46.0	72.0	12.0	4.0	28.0	66.0
codeattacker	24.0	32.0	28.0	38.0	46.0	26.0	18.0	22.0	44.0
pap	52.0	14.0	62.0	70.0	68.0	50.0	48.0	20.0	54.0
gptfuzzer	54.0	2.0	30.0	62.0	12.0	2.0	6.0	18.0	75.0

Attack methods. We compiled a suite of 13 representative attack methods from Table 1, choosing one or two representatives from each category in our taxonomy to ensure broad coverage of the jailbreak space. The selected attacks are LLM-Adaptive [2], ActorBreaker [67], BON (“Best-of-N”) [28], Flip [48], PAIR [12], AutoDAN-Turbo [45], ReNeLLM [18], DrAttack [39], PAP [94], CodeAttacker [66], Multijail [16], CipherChat [93], and GPTFuzzer [91]. For a fair comparison, we cap multi-turn attacks at 20 turns per trial and follow each method’s original parameter settings as reported in the respective papers.

Defense methods. We select five representative defense methods (Table 10) across different categories in the taxonomy, enabling us to evaluate how well each mitigates jailbreak attacks in practice and what trade-offs they introduce. Specifically, our suite includes Hidden State Guard [105], LlamaGuard [29], SelfReminder [80], Aligner [31], and CircuitBreaker [107]. During evaluation, each defense is applied individually to isolate its standalone effectiveness, ensuring that the results clearly reflect the strengths and limitations of each approach.

Judging methods. We evaluate the performance of seven-hao different judge modules. The first is the Rule-Based Judge [108], which relies on predefined rules for evaluation (e.g., starting with “I am sorry”). The second and third are both Prompt-Based LLM Judges: the first is the Score Judge, which assesses harmfulness on a scale from 1 to 10 [2], and the second is the Binary (Safe) Judge [11], which provides a simple success or failure judgment. The fourth and fifth are Fine-Tuned Judges, including LlamaGuard [29] and JailJudge [44], both of which fine-tune an LLM to judge whether something is safe or not. Lastly, the Multi-Agent Judge [44] uses three agents to vote, and the Multi-Level Judge [64] determines jailbreak at different levels.

For our experiments, we primarily use the GPT-4o Score Judge as the final evaluation method. To ensure the robustness of our results, we also employ LlamaGuard, a fine-tuned model known for its effectiveness in detecting jailbreaks while maintaining reasonable time and token costs. In cases where there is disagreement between the judges, human experts are engaged to resolve the discrepancies and

TABLE 13: ASR of three model categories: original older models, fine-tuned older models (trained on HarmBench), and newer models. Red cells indicate the average ASR in fine-tuned older models is more than 10% points higher than that of newer models; Yellow cells indicate within $\pm 10\%$ percentage points; Green cells indicate equal or lower ASR. As shown, newer models generally exhibit a lower average ASR than fine-tuned older models, indicating robustness improvements that extend beyond training data leakage.

Dataset	Attack Method	Original Older Models		Fine-tuned Older Models		Newer Models	
		Qwen	Mistral	Qwen	Mistral	o1-mini	Claude-3.7
HarmBench	llmadaptive	96.0	91.0	72.0	64.0	0.0	0.0
	actorattack	78.5	76.0	60.0	58.0	53.0	26.5
	bon	20.0	46.0	4.0	4.0	3.5	3.0
	flip	49.0	32.0	28.0	2.0	0.0	5.0
	pair	51.0	96.0	50.0	44.0	16.5	12.0
	autodanturbo	52.0	81.0	64.0	64.0	19.5	37.0
	renellm	82.0	85.0	62.0	70.0	68.5	79.5
	drattack	37.0	22.0	12.0	2.0	2.5	2.0
	multijail	35.0	48.0	12.0	10.0	2.0	17.0
	cipherchat	14.0	47.0	38.0	2.0	0.5	42.5
	codeattack	56.0	48.0	18.0	26.0	33.0	40.0
	pap	75.0	80.0	42.0	20.0	10.0	32.0
New Dataset	gptfuzzer	87.5	74.5	6.0	0.0	9.0	0.0
	llmadaptive	88.0	94.0	64.0	92.0	0.0	8.0
	actorattack	76.0	82.0	58.0	70.0	76.0	50.0
	bon	52.0	48.0	4.0	2.0	10.0	10.0
	flip	38.0	22.0	2.0	8.0	2.0	14.0
	pair	92.0	56.0	44.0	52.0	36.0	32.0
	autodanturbo	74.0	88.0	64.0	84.0	36.0	18.0
	renellm	86.0	86.0	70.0	84.0	48.0	36.0
	drattack	22.0	2.0	2.0	0.0	20.0	24.0
	multijail	52.0	24.0	10.0	2.0	6.0	2.0
	cipherchat	35.0	46.0	2.0	0.0	4.0	28.0
	codeattack	40.0	38.0	26.0	46.0	18.0	22.0
	pap	76.0	70.0	20.0	32.0	48.0	20.0
	gptfuzzer	78.0	62.0	0.0	10.0	6.0	18.0

assess the consistency and reliability of the judgments. All reviewers are undergraduate students majoring in computer science, who were given the same instructions as those used in HarmBench for evaluating harmful content. Prior to their participation, they were informed of potential risks, including exposure to offensive content, and were ensured that all data would remain anonymized and used solely for research purposes.

Appendix B. Training Data Exposure

We conducted two experiments to examine the training data exposure problem:

- **Unseen dataset evaluation.** We constructed a new dataset of jailbreak goals designed to have minimal overlap with those in HarmBench (verified via lexical and semantic similarity filtering). This dataset allows us to assess model robustness on tasks that they are highly unlikely to have encountered during training.
- **Exposure simulation.** We fine-tuned two legacy models released before HarmBench, i.e., Qwen2.5-7B and Mistral-7B, directly on HarmBench, aiming to test whether deliberate exposure to benchmark data could reproduce the robustness observed in newer models.

Tables 12 and 13 present the results. We find that:

- On the new dataset, newer models still achieve lower ASR, though with slightly smaller gains (36.0%) than on HarmBench (40.2%), suggesting that robustness improvements generalize beyond the benchmark.

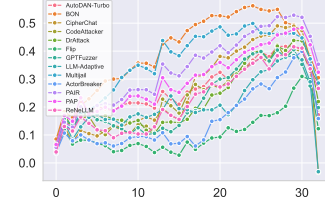


Figure 4: Cosine similarity between successful and failed jailbreak prompts in different model layers. The score fluctuates early, then drops sharply at the last layer, indicating strong separation in the model’s final output space.

- Fine-tuned older models show reduced ASR on both HarmBench (−27.9% on average) and the new dataset (−26.1%), confirming that exposure contributes to robustness. However, their absolute ASR (32.1%) remains substantially higher than that of top newer models (19.8%) on HarmBench.

These findings indicate that while training exposure to benchmark data can enhance robustness, the substantial performance gap between fine-tuned legacy models and the latest models suggests that recent robustness gains primarily stem from improved data curation, multi-stage alignment, and red-teaming processes rather than benchmark leakage.

However, among these newer models, we observe that DeepSeek-v3 and Qwen-3-235B still achieve ASRs as high as 63%, indicating that not all recent releases exhibit guaranteed robustness against jailbreaks. This disparity may stem from several factors, including incomplete adversarial coverage during safety fine-tuning, residual capability, safety trade-offs where highly capable models can inadvertently reason their way around alignment constraints, and insufficient defense-in-depth adaptation for multi-turn or compositional interactions. Put differently, model scale and stronger reasoning capabilities do not inherently ensure safety. Nonetheless, even the most robust models, such as o1-mini and Claude-3.7-Sonnet, remain susceptible to advanced multi-turn or structured attacks (e.g., ActorBreaker [67], ReNeLLM [18]), with ASRs reaching approximately 53–80%. These findings highlight that while modern alignment has substantially improved baseline robustness, lingering architectural and behavioral vulnerabilities persist, underscoring that current alignment methods alone cannot fully eliminate jailbreak risks.

Appendix C. Depth of Disruption

To study depth of disruption in attacks, we provide the figures of cosine similarity of hidden states between successful and failed prompts for each attack method and t-SNE visualization of successful prompts in Figure 4 and 5.

